Software Requirements Specification
FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

October 20, 2026

Contents

Glossary vi

(G) Goals 1

G.1 Context and Overall Objectives . . . 1

G.2 Current situation . . . 1

G.3 Expected Benefits . . . 2

G.4 Functionality overview . . . 3

G.5 High-level usage scenarios . . . 5

G.5.1 Scenario 1: Initial Free-Flight Practice . . . 6

G.5.2 Scenario 2: Completing an Obstacle-Course Level . . . 6

G.5.3 Scenario 3: Recovering from a Simulated Crash . . . 7

G.5.4 Scenario 4: Reviewing Flight Performance . . . 7

G.5.5 Scenario 5: AI-Assisted Flight Feedback (Planned: Subject to Feasibility) . . . 8

G.5.6 Scenario 6: Account Registration and Performance Tracking . . . 10

G.5.7 Scenario 7: Viewing and Comparing Leaderboard Results 10

G.6 Limitations and Exclusions . . . 11

G.7 Stakeholders and requirements sources . . . 12

G.7.1 Stakeholders . . . 12

G.7.2 Requirements Sources . . . 12

G.8 User Profiles . . . 16

G.8.1 Novice FPV Pilot . . . 16

G.8.2 Casual Gamer or Technology Enthusiast 17
G.8.3 Experienced FPV Pilot or Technical Evaluator 17
G.8.4 General User Considerations 18

(E) Environment **19**

E.1 Components 19
E.2 Constraints 26
E.3 Assumptions 28
E.4 Effects 29
E.5 Invariants 31

(S) System **36**

S.1 Components 36
S.2 Functionality 37
S.2.1 Drone Flight Simulation 37
S.2.2 Training Modes and Level Management 39
S.2.3 Checkpoints and Crash Recovery 40
S.2.4 Timing and Performance Evaluation 41
S.2.5 VR Interface and Controller Input 43
S.2.6 Player Accounts and Authentication 44
S.2.7 Result Storage and Leaderboards 45
S.2.8 Post-Flight Review 46
S.2.9 Intelligent Feedback 47
S.2.10 Non-Functional Requirements 47
S.3 Interfaces 50

S.4 Detailed usage scenarios 51
S.5 Prioritization 62
S.6 Verification and acceptance criteria 71
S.6.1 Drone Flight Simulation 71
S.6.2 Training Mode and Level Management 72
S.6.3 Checkpoint and Crash Recovery 73
S.6.4 Timing, Scoring, and Performance Evaluation 75
S.6.5 VR User Interface and Controller Input 77
S.6.6 Player Accounts and Authentication 78
S.6.7 Result Storage and Leaderboards 79
S.6.8 Post-Flight Review 81
S.6.9 Intelligent Feedback 81
S.6.10 Non-Functional Requirements 82
S.6.11 Proposed Quantitative and Interaction Specifications . 85
S.6.12 Proposed Safety, Security, and Failure Behaviour . . . 85

(P) Project **88**

P.1 Roles and personnel 88
P.2 Imposed technical choices 89
P.3 Schedule and milestones 90
P.4 Tasks and deliverables 92
P.5 Required technology elements 95
P.6 Risk and mitigation analysis 97
P.7 Requirements process and report 99

References **103**

Appendix — Generative AI Use Disclosure **104**

Revision History

Date	Developer(s)	Changes
Oct 16, 2026	Sankalpa	Adapted the SRS document structure to Meyer's template.
Oct 18, 2026	Sankalpa	Completed the Goals (G) section and S.2 of the SRS.
Oct 19, 2026	John Cho	Added initial content for System Interfaces and Use Cases (S.1, S.3, S.4)
Oct 19, 2026	Arjun	Completed the Project (P) section of the SRS.
Oct 19, 2026	Vikram	Completed S.5 and S.6 of the SRS.
Oct 19, 2026	Ibrahim	Completed the Environment (E) section of the SRS.

Glossary

The definitions below apply throughout this specification. Each term links to its first use in the text, and every later use of the term links back to this table. Acronyms appearing in figures use these same meanings.

Table 1: Domain glossary and abbreviations

Term	Meaning and reference
FPV	First-person view: presentation from the simulated drone's viewpoint (G.1).
VR	Virtual reality: an immersive presentation viewed through a headset (G.1).
SRS	Software requirements specification: this document's statement of the intended product and its acceptance expectations.
AI	Artificial intelligence: the planned, feasibility-dependent intelligent flight-feedback capability (S.2).
XR	Extended reality: the umbrella term that includes VR; used in the runtime diagrams. It does not specify a particular integration API.
PC / PCVR	Personal computer / PC-based virtual reality: FPVR executes on a computer and presents its VR view through a connected headset (E.1).
Runtime	The external platform software that provides headset presentation, tracking, input, and application-session services (E.1).
Tracking / pose	Tracking reports a device's position and orientation (its pose) and whether that information is currently valid.
Session focus	The runtime's indication of whether FPVR is the active application receiving interaction; visible imagery alone does not prove focus.
Frames per second / hertz	Application frame-delivery rate / display refresh frequency (Hz). These quantities are distinct; the rendering target is specified in S.2.

Continued on next page

Table 1 continued

Term	Meaning and reference
API	Application programming interface: a program-to-program interface. Mention of an external interface does not establish a public FPVR API (S.3).
FR / NFR	Functional requirement / non-functional requirement; prefixes of the existing requirement identifiers in S.2.
Throttle, pitch, roll, yaw	Flight commands for thrust, rotation about the lateral axis, rotation about the longitudinal axis, and rotation about the vertical axis, respectively (S.2).
Checkpoint / segment	A required ordered course gate / the part of a course between the start, successive gates, and the finish (S.2).
Respawn / restart	Resume the same attempt at its most recently completed checkpoint (or start if none) / begin a new attempt from the start (S.2).
Telemetry	Flight and performance measurements used for evaluation. The term alone does not imply recording history or transmitting data (E.4).

List of Tables

1 Domain glossary and abbreviations vi
2 FPVR Functionality Overview 4
3 FPVR Limitations and Exclusions 13
4 FPVR Stakeholders 14
5 FPVR Requirements Sources 15
6 External components and information exchanged 21
7 Controller support for Quest 3S 25
8 Assumptions and how to check them 28
9 Respawn and restart 32
10 Environment traceability and planned checks 34
11 Logical responsibilities and observable contracts 36
12 Priority Assignment: Drone Flight Simulation 63
13 Priority Assignment: Training Modes and Level Management . 64
14 Priority Assignment: Checkpoints and Crash Recovery 65
15 Priority Assignment: Timing and Performance Evaluation . . 66
16 Priority Assignment: VR Interface and Controller Input . . . 67
17 Priority Assignment: Player Accounts and Authentication . . 67
18 Priority Assignment: Result Storage and Leaderboards 68
19 Priority Assignment: Post-Flight Review 69
20 Priority Assignment: Intelligent Feedback 69
21 Priority Assignment: Non-Functional Requirements 70
22 Project milestones and due dates 90

23 Requirement phase-in plan 91
24 Tasks and deliverables 92

List of Figures

1 High-Level FPVR Training Activity Diagram 9
2 FPVR training concepts 20
3 FPVR system context 23
4 Standalone and PCVR modes 25
5 FPVR Logical Component Diagram 38

(G) Goals

G.1 Context and Overall Objectives

Learning to operate a first-person-view (FPV) drone presents challenges for beginners, particularly when practicing with physical equipment. New pilots must develop coordinated flight-control skills while managing the possibility of crashes, equipment damage, and unsafe flight conditions. These challenges can make initial training costly and discourage inexperienced users from gaining practical experience.

FPVR is a virtual reality (VR)-based FPV drone training simulator designed to provide a safe, accessible, and structured environment for developing foundational drone piloting skills. The system will allow users to control a simulated drone, practice flight manoeuvres, and complete progressively challenging courses without requiring a physical drone.

The training environment will include free-flight practice, multiple obstacle-course levels, distinct virtual environments, checkpoints, and performance evaluation. Users will receive feedback on their flight performance through measurable indicators such as completion time, checkpoint progress, and performance ratings. An intelligent feedback component may provide further analysis of user performance.

FPVR will also support registered player accounts, persistent performance records, and local and global leaderboards. Users will be able to practise without signing in, while registered players can save their completed results to an online service, review previous attempts, and compare their performance with other players. These features extend the training experience beyond individual sessions by providing measurable records of progress.

G.2 Current situation

Learning to operate a first-person-view (FPV) drone requires beginners to develop coordinated control skills while interpreting visual feedback from the drone's perspective. Practicing with physical drones introduces risks of crashes, equipment damage, and additional costs, making simulation a

valuable alternative for introductory training.

Several established FPV drone simulators, including Liftoff: FPV Drone Racing [3] and VelociDrone[4], allow users to practice flight manoeuvres, complete racing courses, and explore virtual environments through conventional PC-based interfaces. These platforms provide extensive training and recreational capabilities, with a strong emphasis on realistic flight behaviour, racing, and freestyle flying. However, their traditional desktop experiences do not provide the same interaction environment as a dedicated virtual reality (VR) headset.

VR-based alternatives also exist. Liftoff: XR Drone [5] introduces accessible drone flight using Meta Quest controllers, while DVR Simulator [6] and Drone Blitz [7] offer VR flight challenges and racing or performance-tracking features. These applications demonstrate that immersive drone simulation is already feasible, but they differ in their emphasis on casual exploration, racing, and structured practice.

FPVR aims to build on these existing approaches by focusing specifically on a progressive, beginner-oriented VR training experience. Rather than treating flight practice and course completion as isolated activities, FPVR intends to connect free-flight familiarization, increasingly challenging obstacle courses, checkpoint-based objectives, and post-flight performance reviews into a continuous learning process. By allowing users to evaluate their results and repeat challenges, the system aims to help beginners recognize their mistakes and track improvements over successive attempts.

The intended contribution of FPVR is therefore not the introduction of drone simulation or VR flight itself, but the integration of immersive flight practice with structured progression and measurable feedback. The effectiveness of this approach will require evaluation with potential users.

G.3 Expected Benefits

Safe and Repeatable Practice (G1): FPVR will allow users to practice drone operation without exposing a real drone or surrounding property to collision-related damage. Users can recover from simulated crashes and repeat exercises without the costs associated with damaging physical equip-

ment.

Progressive Skill Development (G2): Rather than relying exclusively on unrestricted flight, FPVR will offer structured challenges of varying difficulty. Users can freely select levels based on their experience and practice increasingly demanding manoeuvres at their own pace.

Performance Awareness (G3): After completing a training level, users will be able to review their performance through recorded measures such as completion time, checkpoint progress, and collisions. This feedback is intended to help users identify areas for improvement and encourage deliberate practice.

Accessible Introductory Training (G4): FPVR will accommodate users with limited or no prior FPV flying experience by providing a free-flight environment where they can become familiar with flight controls before attempting structured challenges.

Persistent Progress and Performance Comparison (G5): FPVR will allow registered users to retain their completed flight results across sessions and access them from supported installations. Local and global leaderboards will provide opportunities to compare performance on individual obstacle-course levels, encouraging users to revisit challenges and track their improvement over time.

These benefits concern the intended training experience. Any claim that FPVR improves actual physical-drone piloting ability will require separate validation and is not assumed by this specification.

G.4 Functionality overview

FPVR will provide the following capabilities to support its goals.

Table 2: FPVR Functionality Overview

Functionality	**Description**	**Initial Scope**
VR Flight Experience	First-person simulated drone flight through a virtual reality headset.	Core
Free-Flight Practice	An open-ended practice environment where users can freely operate the drone without course objectives, timers, or scoring.	Core
Obstacle-Course Levels	Structured flight challenges of varying difficulty. Users may select any available level regardless of previous completion.	Core
Different Environments	Multiple virtual environments for flight practice and obstacle courses. Different levels may share the same environment.	Core
Checkpoint System	Designated course checkpoints used to track progress toward level completion.	Core
Crash Recovery	Following a simulated crash, users can choose to restart the level or respawn at the most recently reached checkpoint.	Core
Timing and Scoring	Records completion time and calculates a score based on defined performance measures during obstacle-course challenges.	Core
Flight Review	Provides a summary of flight performance after completing a level, including completion time, collisions, checkpoint results, and score.	Core
Flight Rating	Provides a separate performance rating based on defined flight measures. The evaluation criteria remain to be determined.	Core

Functionality	Description	Initial Scope
Intelligent Feedback	Potential AI-based analysis of flight performance to provide additional observations and recommendations, subject to feasibility evaluation.	Planned
Controller Input	Supports drone control using Meta Quest controllers or a compatible Xbox controller.	Core
Player Accounts	Allows users to register, sign in, and sign out using a username and password. Users may continue basic flight training without an account.	Core
Cloud Result Storage	Stores completed flight results for registered players, allowing synchronized results to be accessed across supported installations.	Core
Local and Global Leaderboards	Provides level-specific rankings using locally saved results and results submitted by registered players to the online service.	Core
Performance History	Allows registered users to review results from previously completed obstacle-course attempts.	Core

The exact scoring and rating criteria, controller mappings, checkpoint and respawn behaviour, and requirements for AI-based feedback will be specified in the System section.

G.5 High-level usage scenarios

This section describes the primary interactions between users and FPVR during flight practice, obstacle-course completion, crash recovery, and performance evaluation. These scenarios illustrate the intended system behaviour and support the goals defined in Section G.1.

G.5.1 Scenario 1: Initial Free-Flight Practice

A first-time user launches FPVR and selects the free-flight mode using either Meta Quest controllers or a compatible Xbox controller. The user enters a virtual environment where they can practice basic flight manoeuvres, such as ascending, descending, turning, and navigating through the environment.

Unlike structured obstacle-course levels, free-flight mode has no checkpoints, timers, or scoring. Users can explore and practice at their own pace without specific completion objectives.

If the drone experiences a simulated crash, the user can reset its position and continue practicing. The user may leave free-flight mode and return to the level-selection interface when desired.

This scenario supports G1 and G4 by providing a safe, repeatable environment for becoming familiar with FPV drone controls.

G.5.2 Scenario 2: Completing an Obstacle-Course Level

A user selects an obstacle-course level from the available challenges. All levels are accessible regardless of previous completion, allowing users to choose courses appropriate to their experience and preferences.

Each level contains obstacles and a sequence of checkpoints that must be completed in the designated order. The system tracks the user's progression, total elapsed time, and time spent navigating between checkpoints. Upon reaching a checkpoint, the current section timer resets while the total elapsed time continues accumulating.

The user navigates through the checkpoints in sequence until reaching the finish. After successful completion, the system calculates a numerical score and a separate performance rating based on the recorded flight results.

The user can then review their performance, retry the level, or select another challenge.

This scenario supports G2 by providing structured flight challenges of varying difficulty with measurable outcomes.

G.5.3 Scenario 3: Recovering from a Simulated Crash

While attempting an obstacle-course level, the user loses control of the drone and experiences a simulated crash. The system records the collision and provides two recovery options: restart the entire level or respawn at the most recently reached checkpoint.

If the user selects respawn, the drone returns to the last successfully reached checkpoint. If no checkpoints have been reached, the drone returns to the level's starting position.

The current section timer resets upon respawning, allowing the user to attempt the section again. However, the total elapsed time continues accumulating, including the time spent on failed attempts. The recorded collision also negatively affects the final score.

The user can continue attempting the course without restarting the FPVR application.

This scenario supports G1 and G2 by allowing users to recover from mistakes while maintaining meaningful performance consequences.

G.5.4 Scenario 4: Reviewing Flight Performance

After successfully completing an obstacle-course level, the user receives a flight review summarizing their performance.

The review includes the total completion time, checkpoint results, collision count, and numerical score. The total completion time includes time spent on failed sections and subsequent attempts following checkpoint respawns.

The system also presents a separate overall performance rating based on defined evaluation criteria. The user can examine these results to identify areas for improvement before attempting another flight challenge.

This scenario supports G3 by providing measurable feedback intended to encourage deliberate practice and improvement.

The exact scoring formula, rating categories, and penalties associated with collisions must be established in S.2.

G.5.5 Scenario 5: AI-Assisted Flight Feedback (Planned: Subject to Feasibility)

If AI-assisted feedback is included in the final implementation, the system will automatically provide additional performance analysis as part of the post-level flight review.

Using available flight-performance data, the system would identify areas of difficulty and provide understandable observations or recommendations. For example, the feedback might highlight repeated collisions or difficulties navigating particular obstacle sections.

The intelligent feedback component is intended to complement the standard numerical score and performance rating rather than replace them.

The feasibility, data requirements, analysis approach, and evaluation criteria for AI-assisted feedback remain to be determined. The standard flight review will remain available independently of this optional capability.

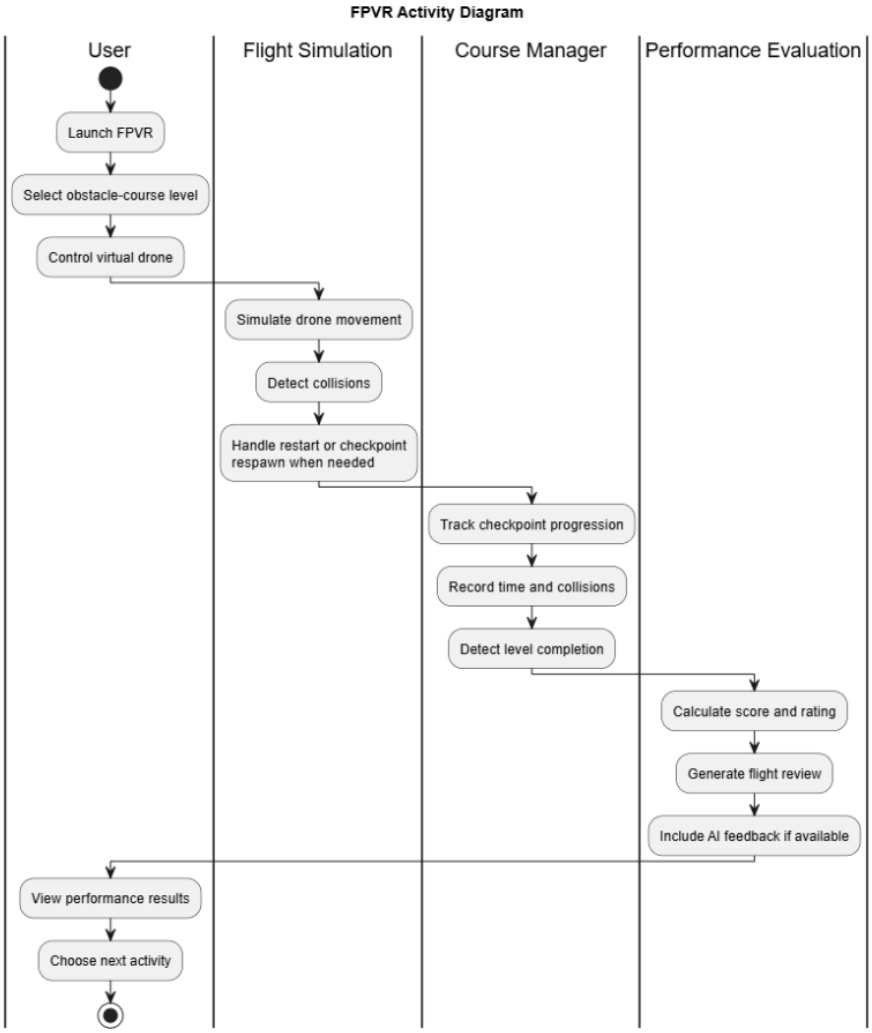


Figure 1: High-Level FPVR Training Activity Diagram

The activity diagram illustrates the primary obstacle-course training workflow. Swimlanes identify the responsible user or system component, while arrows indicate the sequence of activities.

G.5.6 Scenario 6: Account Registration and Performance Tracking

A user launches FPVR and chooses to register a player account using a username and password. After successfully registering or signing in, the user selects an obstacle-course level and completes a flight challenge.

Upon completion, FPVR presents the standard post-flight review, including the completion time, collision count, numerical score, and performance rating. The result is saved locally and, when the online service is available, synchronized with the user's registered account.

The user may later sign in on another supported installation to retrieve previously synchronized results and review their performance over multiple attempts.

This scenario supports G3 and G5 by enabling persistent performance records and allowing users to monitor their improvement over time.

G.5.7 Scenario 7: Viewing and Comparing Leaderboard Results

After completing an obstacle-course level, a user opens the leaderboard for that level.

FPVR displays a local leaderboard containing results saved on the current installation. When the online service is available, the user can also view a global leaderboard containing results submitted by registered players.

The leaderboards rank results according to numerical score, with completion time and completion timestamp used to resolve ties. Users can compare their performance with other recorded attempts and repeat the level to improve their ranking.

If the online service is unavailable, the local leaderboard remains accessible, and the user can continue training normally.

This scenario supports G3 and G5 by providing measurable performance comparisons and encouraging repeated practice.

G.6 Limitations and Exclusions

FPVR is intended to provide an introductory virtual reality (VR) training experience for first-person-view (FPV) drone operation. The following limitations and exclusions define the boundaries of the system and clarify functionality outside the project's current scope.

FPVR focuses on safe introductory flight practice, structured obstacle-course challenges, and performance evaluation within a virtual environment. The system is not intended to replace real-world flight training or serve as a comprehensive engineering-grade flight simulator. The risks associated with these limitations and their corresponding mitigation strategies are discussed in P.6.

G.7 Stakeholders and requirements sources

This section identifies the stakeholders who may influence or be affected by FPVR and the sources used to establish its initial requirements. It distinguishes confirmed project participants from potential stakeholders whose feedback has not yet been collected.

G.7.1 Stakeholders

Table 4 identifies the primary and secondary stakeholders of FPVR and their interests in the project.

Society is considered an indirect stakeholder because VR-based drone training has implications for physical safety, user well-being, accessibility, and responsible drone operation. FPVR must not misrepresent simulated flight performance as proof of real-world piloting competence.

FPVR is intended for users with varying levels of experience and age, subject to the age restrictions and safety requirements of the supported VR hardware. Applicable platform age-group requirements are documented by Meta [10].

G.7.2 Requirements Sources

The initial requirements for FPVR have been developed using existing project documentation, team discussions, supervisor guidance, and technical references relevant to virtual reality and flight simulation.

The principal requirements sources are summarized in Table 5.

Table 3: FPVR Limitations and Exclusions

Area	**Limitation or Exclusion**
Physical Drone Operation	FPVR is limited to simulated drone operation and will not communicate with or control physical drones.
VR Hardware Requirement	FPVR requires a compatible VR headset for operation. Both Meta Quest controllers and compatible Xbox controllers will be supported, but conventional PC gameplay without a VR headset is excluded.
Pilot Certification	FPVR will not provide official drone pilot certification or replace applicable regulatory examinations, assessments, or licensing requirements.
Real-World Flight Proficiency	Performance within FPVR will not be considered proof of a user's ability to safely operate a physical drone. The transfer of skills from simulation to real-world flight is not guaranteed.
Flight Simulation Realism	FPVR aims to provide realistic flight behaviour for training purposes. However, the underlying flight model, its implementation, and the required level of accuracy remain under review. The system is not intended to reproduce every physical and environmental characteristic of real-world drone operation.
Multiplayer Functionality	The core FPVR system is limited to single-player training. Multiplayer flight, in which multiple users operate drones in a shared virtual environment, is a stretch goal and will be implemented only if development time permits (LC-08).
Dedicated FPV Transmitters	Support for dedicated FPV radio transmitters is a stretch goal and is not part of the core controller requirements (FR-UI-03). Compatibility and feasibility will be evaluated during development (LC-08).
Online Service Dependency	Account registration, global leaderboards, and cloud synchronization require network connectivity and an available online service. Core training activities, local result storage, and standard post-flight reviews remain available without these online services.
AI-Assisted Feedback	AI-generated flight feedback is planned subject to feasibility evaluation. Core flight reviews, numerical scores, and performance ratings will remain available independently of this optional capability.

Table 4: FPVR Stakeholders

Stakeholder	**Interest in FPVR**
Beginner FPV Pilots	Primary intended users seeking accessible introductory training, opportunities to practise basic flight manoeuvres, and feedback on their performance.
Drone Enthusiasts	Potential users interested in additional flight practice, structured challenges, and evaluating the realism of the simulation.
Students and Technology Enthusiasts	Potential users interested in experiencing FPV drone operation through virtual reality. Intended use is subject to applicable headset age and safety restrictions.
McMaster Drone Team	Potential technical stakeholder that may provide feedback on simulated drone behaviour and operator training. The team has not yet been consulted.
Project Team	Responsible for developing, testing, maintaining, and documenting FPVR according to the established requirements.
Project Supervisor	Provides technical guidance and feedback on project scope, feasibility, and simulation realism.
Course Evaluators	Review the project's requirements, engineering process, documentation, and delivered functionality against the course expectations.
Parents and Guardians	Indirect stakeholders concerned with age-appropriate access and the safe use of VR by younger users.
Society and General Public	Indirect stakeholders interested in safer introductory drone training, responsible technology use, accessibility, and the management of VR-related health and safety concerns.
Registered Players	Users who create accounts to preserve completed flight results, review their performance history, and participate in global leaderboards. Their interests include reliable result storage, account security, and access to synchronized performance information.

Table 5: FPVR Requirements Sources

Source	**Contribution to Requirements**
Problem Statement	Establishes the problem being addressed, the project's objectives, and its intended benefits.
Development Plan	Provides project scope, planned development activities, technical considerations, and project constraints.
Hazard Analysis	Identifies potential hazards and safety considerations relevant to the simulator and its users.
Supervisor Discussions and Meeting Notes	Provide technical guidance on simulation realism, drone flight behaviour, and the selection of an appropriate flight model.
Unity Documentation	Provides information about available physics simulation capabilities, including movement, forces, gravity, and collision handling [8].
Meta Quest Documentation	Provides technical information about VR device integration and controller input [9], as well as applicable age-group requirements [10].
Existing FPV Simulators	Provide a basis for comparing existing flight practice, training, and performance evaluation capabilities with the proposed functionality of FPVR.
Potential End Users	Planned source of feedback on usability, flight-control expectations, training needs, and the effectiveness of performance feedback. Direct consultation has not yet occurred.

The project supervisor has provided guidance concerning realistic flight behaviour and the selection of an existing drone model as a possible basis for the simulation. These discussions have informed the identification of technical concerns, but the final flight model and required accuracy remain under review.

No structured requirements-elicitation sessions have yet been conducted with the intended end users. The project team plans to consult potential users, including beginners and individuals familiar with drone operation, to validate assumptions about usability, training needs, and performance feedback.

Until this consultation takes place, the identified user needs and evaluation targets remain provisional. Feedback collected during development will be used to review and refine the system requirements.

G.8 User Profiles

FPVR is intended for users with varying levels of experience in drone operation, virtual reality (VR), and video games. The system primarily targets beginners while also accommodating recreational users and experienced FPV pilots. Users must comply with the age and safety restrictions of the supported VR hardware.

The following profiles describe the anticipated experience, needs, and usage patterns of the system's main user groups.

G.8.1 Novice FPV Pilot

The primary user is an individual with little or no experience operating first-person-view (FPV) drones. This user may also be unfamiliar with VR environments and their associated controllers.

Experience: May have general familiarity with computers or video games but limited knowledge of drone flight controls, navigation, and flight behaviour.

Needs: Clear control instructions, understandable interactions, opportunities to practise without risking damage to physical equipment, and feedback

on flight performance.

Expected Usage: The user reviews the basic control instructions and begins practising in free-flight mode. As confidence improves, the user may select obstacle-course levels of increasing difficulty. Progression is voluntary, as all levels are available without prerequisite completion. The user may initially practise without creating an account. If they choose to register, they can preserve their completed results and review previous attempts to evaluate their progress over time.

G.8.2 Casual Gamer or Technology Enthusiast

This user is interested in experiencing drone flight through VR without necessarily intending to purchase or operate a physical drone.

Experience: May be familiar with video-game controllers and immersive experiences but have limited knowledge of actual FPV drone operation.

Needs: Engaging virtual environments, accessible controls, understandable objectives, and opportunities to measure improvement through repeated challenges.

Expected Usage: The user explores free-flight environments and attempts obstacle-course challenges for recreation and exploration. The user may repeat levels to improve completion times, scores, and performance ratings. The user may also create an account to participate in global leaderboards and compare their results with other players. Local leaderboards provide an additional way to compare attempts recorded on the same installation.

G.8.3 Experienced FPV Pilot or Technical Evaluator

This user has prior experience operating physical FPV drones or working with related technical systems. Experienced users may participate in regular training activities or provide feedback during system evaluation.

Experience: Familiar with drone controls, flight manoeuvres, and the expected behaviour of physical drones.

Needs: Consistent control responses, meaningful obstacle-course challenges, and sufficient performance information to evaluate the simulator's behaviour.

Expected Usage: The user practises in free-flight mode, completes obstacle-course challenges, and reviews recorded performance results. The user may also compare simulated flight behaviour with prior experience and identify areas for improvement. The required accuracy of the underlying flight model remains under review. Registered experienced users may also review historical results and compare their performance across repeated attempts. These comparisons may support evaluation of the simulator's consistency, but do not establish real-world flight proficiency.

G.8.4 General User Considerations

FPVR is intended to accommodate both seated and standing operation using a compatible VR headset. Users may interact with the simulator through Meta Quest controllers or compatible Xbox controllers.

Users may differ in physical capabilities, experience with immersive environments, and susceptibility to VR-related discomfort. The system's design will consider applicable headset safety guidance and the needs of different user groups.

The profiles presented in this section are based on the intended target audience rather than completed user interviews. As discussed in Section G.7, the project team plans to consult potential users to validate their needs, expectations, and usage assumptions.

(E) Environment

This section describes where FPVR runs and how it interacts with people and hardware. It follows Meyer's Environment template [1], with the glossary placed after Revision History. Detailed software behaviour is defined in S.2.

Identifiers beginning with **E-C**, **E-A**, **E-F**, and **E-I** identify constraints, assumptions, effects, and invariants. Assumptions must be checked; invariants are rules FPVR must always preserve.

E.1 Components

FPVR includes the simulated drone, flight spaces and courses, crash recovery, performance results, player accounts, global leaderboards and VR controls. The headset, controllers, connected computer, authentication service, cloud database and online leaderboard service are outside the software boundary. G.6 describes the simulation limits.

Domain relationships

Figure 2 shows the main training concepts. A course takes place in one virtual environment and contains ordered checkpoints. Each attempt uses one course and one simulated drone. Completing the attempt produces a flight review. Registered users can save and retrieve completed results. The diagram shows one course run; it does not specify software classes.

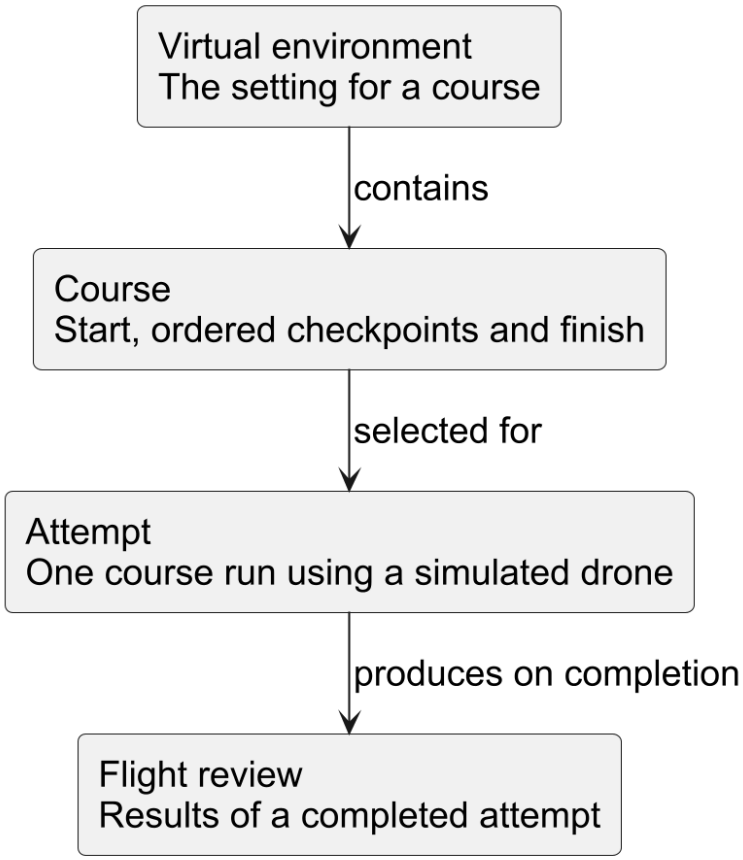


Figure 2: FPVR training concepts. Read from top to bottom; each arrow names the relationship between the connected concepts.

External components

Users, their physical space, input devices, and the headset/runtime affect FPVR. PCVR also uses a computer and a headset link. Authentication, cloud storage and online leaderboard services support accounts and saved results. Guardians and nearby people are affected stakeholders (G.7); user profiles are in G.8. Table 6 explains the exchanges labelled F1–F8 in Figure 3.

Table 6: External components and information exchanged

Component / flow	**Inputs to FPVR**	**Outputs from FPVR**
User (F1)	Account, mode, level, menu, recovery and exit choices; flight controls via F3.	Flight view, instructions, recovery choices, current and saved results, and rankings.
Headset and runtime (F2)	Device position/orientation (pose), tracking validity, input availability, session focus, display timing and session status.	Rendered images and responses to runtime services.
Quest or compatible Xbox controller (F3)	Controller type, flight axes, buttons and connection state.	Instructions and device feedback shown to the user through F1.
PCVR computer and headset link (F4)	Computer resources, runtime/link status and headset tracking/input.	PC-rendered images sent to the headset; PCVR only.
Optional analysis (F5)	Additional feedback or a failure/unavailable result.	Flight data for analysis, if included. Data and processing location remain under review (S.2).
Authentication service (F6)	Registration/login outcomes and authenticated session state.	Account details, login credentials and sign-in/sign-out requests.

Component / flow	**Inputs to FPVR**	**Outputs from FPVR**
Cloud database (F7)	Saved flight results and save/retrieval status.	Completed flight results linked to the user account, and requests for previous results.
Online leaderboard service (F8)	Global rankings and score-submission status.	Course scores, player identifiers and ranking requests.
Physical space	Room clearance, user preparation and conditions for headset tracking. No direct obstacle-sensing input is assumed.	VR images affect user attention and movement; FPVR cannot clear the physical space.

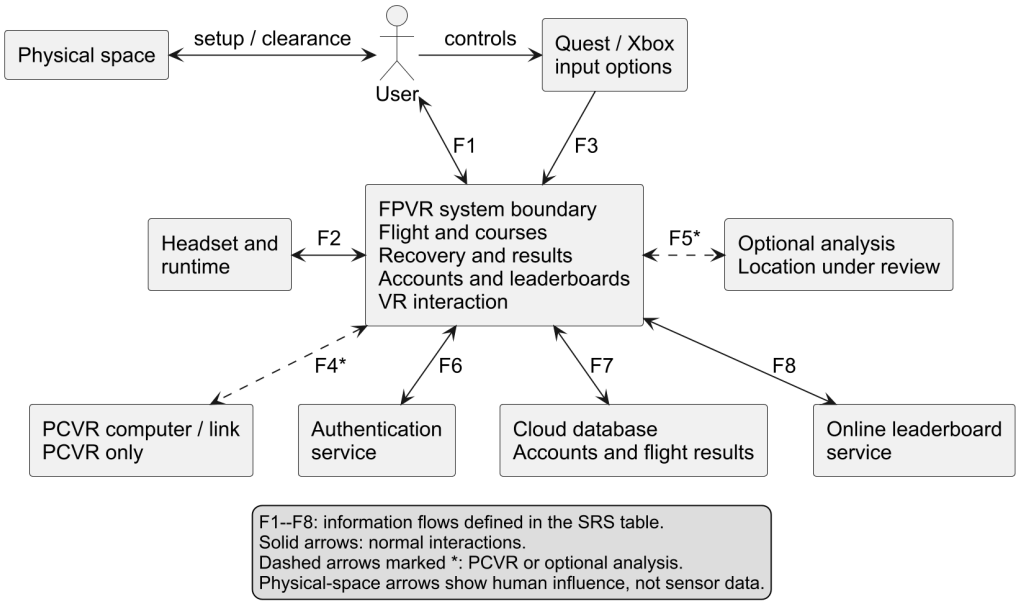


Figure 3: FPVR system context. F1–F8 refer to Table 6. Arrowheads show direction. Solid arrows show normal interactions; dotted arrows marked * apply only to PCVR or optional analysis. Arrows between the user and physical space show preparation and physical influence, not a sensor connection.

User-facing images and results pass through the headset. F4 applies only to PCVR. F5 is optional analysis; F6–F8 are required online services for accounts, saved results and global leaderboards. These online features need internet access in either execution mode. Detailed interfaces belong in S.3.

Execution environments and compatibility

FPVR must support both modes below. A session uses one mode at a time (NFR-INST-01–02).

- **Standalone Quest:** FPVR runs on the headset and uses its display, tracking and input services. No connected computer is needed during play.
- **PCVR:** FPVR runs on a computer. A compatible runtime and headset link send images to the Quest headset and return tracking and input data.

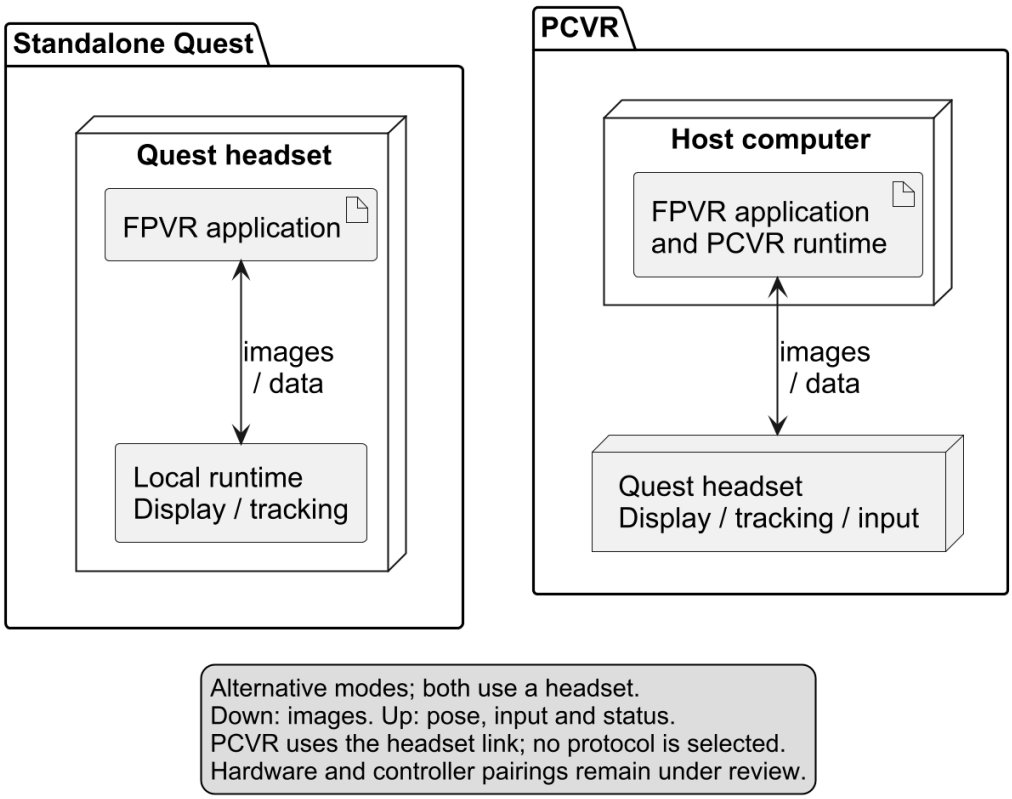


Figure 4: Standalone and PCVR modes. Outer boxes separate the modes; inner boxes show where FPVR runs and where images are displayed. Arrows show the named information flows. Both modes use a headset; the link technology is still to be chosen.

Table 7: Controller support for Quest 3S

Controller type	**Support commitment**	**Execution-mode compatibility**
Quest motion controllers	Required	Standalone/PCVR pairings to validate
Xbox-compatible controllers	Required	Standalone/PCVR pairings to validate

FR-UI-02–03 and NFR-COMP-01 require a working configuration for each controller type across the system on the confirmed Quest 3S target. Table 7 records these commitments; individual standalone and PCVR pairings still need validation. For each tested pairing, record the controller, runtime and control mapping; PCVR also needs the computer operating system, hardware and link details (E-C-02).

E.2 Constraints

Constraints are limits the project must follow. The items below come from the project scope, chosen platforms and release conditions.

E-C-01: Platforms and engine. FPVR must support the two modes and controller types described in E.1 (FR-UI-01–03, NFR-INST-01–02, NFR-COMP-01). The Development Plan, "Expected Technology", selects Unity. *Reason:* these choices support immersive practice using the planned hardware and development tools.

E-C-02: Supported hardware. Quest 3S is the finalized target headset. Support for Quest motion controllers and Xbox-compatible controllers is required overall. Specific Xbox controller versions, computer requirements, headset links and execution-mode pairings still need validation. Record tested configurations as explained in E.1 (NFR-COMP-01). *Reason:* choosing a device does not prove that FPVR works on it.

E-C-03: Scope and flight accuracy. The simulation limits in G.6 still apply. Player accounts, cloud-backed results and global leaderboards are required. Simulated gravity is required by FR-FLT-03; the required level of physical accuracy is still under review. *Reason:* the Problem Statement aims at safe introductory practice. Simulator results alone cannot establish real-world piloting ability.

E-C-04: Safety and intended users. Use must follow the selected headset's safety instructions, including space preparation, appropriate supervision and breaks. Age and regional eligibility must be reviewed for the users described in G.8. *Reason:* VR affects a person in a real room.

Stationary controls and an accessible exit (FR-UI-06, NFR-SAFE-01) support safer use but cannot ensure comfort for every user.

E-C-05: Release and data rules. A Meta release must meet the applicable audience rules. Distribution must also respect engine, runtime and asset licences. Account data, completed flight results and leaderboard entries require persistent cloud storage. Protect credentials and stored data, encrypt service connections, and restrict access to private records to their owner and authorised services. Define the data collected, its purpose, recipients, retention period, deletion process and user notice/consent and applicable legal duties before release. Explain which leaderboard details are public. Optional AI processing must have a separate data-use policy; storing results does not authorise sending them or credentials to an AI service. *Reason:* required storage supports accounts, previous results and global rankings, while optional analysis (S.2) has a separate purpose.

E-C-06: Hardware limits. Standalone flight depends on headset processing power, memory, battery and heat limits. PCVR also depends on the computer and headset link. The proposed minimum rendering target is 72 frames per second during normal flight and obstacle-course operation on supported hardware (NFR-PERF-01). *Reason:* these limits affect smooth, responsive flight. The target needs validation on Quest 3S and the supported PCVR configurations; computer requirements remain unresolved in E-C-02.

External sources

The following sources support the safety and release constraints. Check the current guidance for the selected headset and release.

Headset safety. Meta Platforms, Inc., *Meta Quest Safety Center*, model-specific safety guidance. Accessed October 19, 2026. Supports E-C-04 and E-A-02.

Audience policy. Meta Platforms, Inc., *Age group self-certification policy*, Meta Horizon OS developer policy. Accessed October 19, 2026; see also [10]. Supports E-C-05.

E.3 Assumptions

These assumptions describe the conditions needed for use and testing. Each row explains how to check the assumption and what happens if it fails.

Table 8: Assumptions and how to check them

Identifier	Assumption and reason	Check and result if false
E-A-01	The chosen hardware has a compatible runtime that reports headset, controller and session status. FPVR needs these services to run in VR.	Launch and use proposed pairings of the controller types in Table 7; record versions and status changes. A pairing with missing or incompatible services cannot be accepted (E-C-02).
E-A-02	The user or guardian can prepare a suitable stationary space using the headset instructions. This allows seated or standing use.	Review setup and try the stationary controls and exit (FR-UI-06, NFR-SAFE-01). Poor clearance or discomfort means safe-use conditions are not met; FPVR cannot certify that the room is safe.
E-A-03	Each released course has a valid start, ordered checkpoints, finish, collision surfaces and usable recovery positions. These make progress and recovery meaningful.	Inspect and fly each course, including recovery before the first checkpoint. Invalid or unreachable course content must be corrected before acceptance (FR-CHK-01–08).
E-A-04	Tests can access the claimed hardware pairings and representative free-flight and course content. These are needed to check compatibility and performance.	Record the equipment and courses used. Missing equipment or content leaves its checks unverified and its pairing unconfirmed (NFR-COMP-01, NFR-PERF-01).

Tracking, focus and PCVR links can fail. Their failure handling still needs

specification and checks in S.3, S.4, and S.6. An available AI service is not assumed; the standard review must remain available as explained in E-I-04. Account login, cloud save/retrieval and leaderboard requests depend on internet access and their services. Check service failures and rejected credentials; show unsuccessful requests clearly and do not report failed saves as saved.

E.4 Effects

Effects describe how FPVR affects users and equipment. They also concern guardians, bystanders and the wider public (G.7) through activity-space safety and training claims.

E-F-01: Comfort and attention. The moving drone view can cause discomfort or reduce attention to nearby people and objects. The user remains in a physical room even though the drone and its collisions are simulated. Safety guidance, space preparation and an accessible exit address these concerns (E-C-04, E-A-02, NFR-SAFE-01).

E-F-02: Practice and feedback. Repeated challenges and flight results are intended to help users recognize mistakes (G.3, FR-REV-01–03). A high score may also give a user too much confidence about real flying. Results describe simulator performance; any improvement in real-world skill needs separate evaluation (G.6).

E-F-03: Flight data and equipment use. FPVR updates the selected course, checkpoint progress, timers, crash count, score, rating and current review (FR-PER-01–10, FR-REV-02). Registered users can save completed results to persistent cloud storage and retrieve previous results. Account-linked records and public leaderboard entries create privacy and access-control obligations under E-C-05. FPVR also uses device processing power, memory, display, battery and network access; PCVR uses the computer and link (E-C-06).

E-F-04: Optional analysis. If included, intelligent analysis adds observations about the completed attempt. It may run locally or use an external service; this choice is still under review. An external service

would add the F5 connection in Figure 3, subject to E-C-05. The standard review stays available without this analysis (E-I-04).

E.5 Invariants

An invariant is a rule that must stay true while FPVR operates. Values such as elapsed time and checkpoint count can change, but they must change according to these rules. The rules below explain the required behaviour and link to the requirements that define it.

E-I-01: Simulated flight only. FPVR controls a simulated drone. No action, including moving the throttle or recovering from a crash, may send a command to a physical aircraft. A score or rating also does not certify real-world piloting ability (G.6). *Reason:* users need to understand the limits of simulator practice.

E-I-02: Checkpoint order and recovery. Completed checkpoints must always form the beginning of the course's required sequence, with none skipped. Respawn keeps progress in the same attempt; restart begins a new attempt.

Checkpoint order. Let n be the number of required checkpoints (a positive integer), listed in order as $\langle C_1, \ldots, C_n \rangle$. Let k count the completed checkpoints and P list them. Then

$$k \in \{0, \ldots, n\}, \qquad P = \langle C_1, \ldots, C_k \rangle.$$

This says that k cannot be below zero or above n, and P contains exactly the first k checkpoints. Before the first checkpoint, $k = 0$ and P is empty.

During active flight, when no crash awaits recovery, let e be a checkpoint passage. A prime ($'$) means the value just after that event:

$$k' = \begin{cases} k+1, & k < n \text{ and } e \text{ is passage through } C_{k+1}, \\ k, & \text{otherwise.} \end{cases}$$

Only the next required checkpoint increases progress. Repeated or out-of-order checkpoints do not count, and progress cannot advance while a crash awaits recovery (FR-FLT-06, FR-CHK-01–03). Reaching the finish counts as completion only after all n checkpoints are complete (FR-CHK-08).

Example, using three checkpoints: after C_1, $k = 1$ and $P = \langle C_1 \rangle$. Passing C_3 next changes neither value. Passing C_2 then gives $k = 2$ and $P = \langle C_1, C_2 \rangle$. The user must still pass C_3 and reach the finish. These numbers illustrate the rule; they do not set the number of checkpoints in every course.

Crash recovery. Respawn continues the current attempt; restart starts again from the beginning. Table 9 shows exactly what is kept or reset (FR-CHK-05–07).

Table 9: Respawn and restart

Item	Respawn	Restart
Attempt	Continue the same attempt	Begin a new attempt
Completed checkpoints	Keep k and P	Set $k = 0$ and clear P
Drone position	Last completed checkpoint; start if $k = 0$	Course start
Total flight time	Keep accumulated time	Reset to zero
Current section time	Reset to zero	Reset to zero
Crashes and scoring	Keep crash count and recorded penalties	Reset crash count and performance score

Example: checkpoints C_1 and C_2 are complete, total flight time is 42 seconds, and one crash has been counted. Respawn returns the drone to C_2 and keeps both checkpoints, the 42 seconds, the crash count and its penalty. The current section timer resets to zero. These values illustrate recovery; they are not performance targets.

Timing. Failed flight still counts toward total time. Time spent in the crash-recovery menu does not. Let T be total flight time and Δt be a non-negative elapsed interval, both measured in seconds. The total after that interval is T':

$$T' = \begin{cases} T + \Delta t, & \text{during active flight,} \\ T, & \text{while the crash-recovery menu is displayed.} \end{cases}$$

For example, spending ten seconds in that menu does not add ten seconds to T. Respawn keeps T; restart resets it. The section timer resets when the next checkpoint is completed or the drone respawns.

Each new crash adds one to the crash count (FR-PER-01–07). Total time stops at successful completion (FR-PER-10). *Reason:* respawn must not erase failed-flight time or crash penalties; restart begins a new attempt. These course rules do not apply to free flight, which has no checkpoints, timers or score.

E-I-03: Stationary interaction. Users must be able to control flight, use menus, recover and exit while seated or standing. Flying through a virtual course must not require walking through the physical room. Exit must be accessible without removing the headset (FR-UI-06, NFR-SAFE-01). *Reason:* the controls must suit the prepared stationary space, including after a simulated crash.

E-I-04: Standard review without optional analysis. After successful completion, the user must receive the current attempt's time, checkpoint results, crash count, score and rating. This review remains available whether intelligent feedback is not included, succeeds, is unavailable or fails. For example, an analysis error must still allow the user to review the results and retry the course (FR-REV-01–04, FR-AI-03). *Reason:* an optional feature must not block the core training experience.

Traceability and checks

Each Environment ID links to a check below, and each row links back to the rule. The middle column links to the related scope or requirement. These are planned checks, not completed software or user tests.

Table 10: Environment traceability and planned checks

Rule IDs	**Related scope / requirement**	**Planned check**
E-C-01, E-C-02, E-A-01, E-A-04	FR-UI-01–03; NFR-INST-01–02, NFR-COMP-01	Launch both modes and test each claimed controller pairing. Record hardware, runtime, mapping and PCVR link details. Mark untested pairings and check the chosen engine in the build.
E-C-03, E-F-02, E-I-01	G.3, G.6; FR-FLT-03	Check outputs for physical-drone commands and result text for certification or unverified training claims. Check simulated gravity against approved criteria.
E-C-04, E-A-02, E-F-01, E-I-03	FR-UI-06; NFR-SAFE-01; headset guidance	Review headset setup. Try flight, menus, recovery and exit while seated and standing without walking. Record comfort and accessibility issues.
E-C-05, E-F-03	S.2; Optional analysis	Test registration/login, cloud save/retrieval of previous results and global rankings (F6–F8), including service failures. Check access to another user's private results is denied. Review encryption, retention/deletion, public leaderboard fields, separate AI data use, licences and audience rules.
E-C-06	NFR-PERF-01	Measure frame delivery against the proposed 72 FPS minimum and record hardware/link behaviour on supported configurations using a repeatable course workload.

Rule IDs	Related scope / requirement	Planned check
E-A-03, E-I-02	FR-CHK-01–08; FR-PER-01–07, FR-PER-10	Try ordered, repeated and skipped gates, early finish and gate passage while crashed. Crash before and after a checkpoint; check Table 9, menu-time exclusion and the completion timer.
E-F-04, E-I-04	FR-REV-01–04; FR-AI-03	Complete a course without optional feedback. If implemented, also try available, unavailable and failed feedback. Check the standard review and retry option in each case.

Likely changes. Supported devices, runtime/link choices, performance limits and optional data processing may change as they are tested (E-C-02, E-C-05, E-C-06). The four invariants are expected to stay stable because they protect the simulator's scope and core training experience (E-I-01–E-I-04).

(S) System

S.1 Components

The following are logical responsibilities for specifying behaviour, not a module design: (1) drone flight and collision; (2) training modes and level selection; (3) checkpoint progression and recovery; (4) timing and evaluation; (5) VR interfaces and controller input; (6) post-flight review; (7) account registration and leaderboard access; and (8) optional intelligent feedback. Each has a corresponding subsection of S.2. The first seven comprise the core. The eighth is conditional on a separate feasibility and scope decision. Safety, validation, and deployment properties apply across these responsibilities.

Table 11: Logical responsibilities and observable contracts

Responsibility	Observed inputs	Observed outputs
Flight/collision	Flight axes, gravity, contact, current pose	Drone view, movement, one latched crash event
Modes/levels	Practice/course selection, exit/restart	Selected environment, initialized or ended attempt
Progress/recovery	Gate crossing, finish, recovery choice	Ordered prefix, recovery pose, completion state
Timing/evaluation	Active interval, checkpoint/crash/completion events	Total and segment time, crash count, score/rating
VR interfaces	Pose/focus/input availability and user action	Instructions, menus, flight/review presentation
Review/optional AI	Final current-attempt result, optional analysis outcome	Standard review, optional labelled feedback or fallback
Accounts/leaderboards	Registration, sign-in, saved result, ranking request	Account status, local ranking, global ranking or availability status

The FPVR system is a VR-based first-person-view (FPV) drone simulation game. Its major components are the VR interface, the flight simulation, the virtual environment, the training and scenario manager, and the telemetry

and feedback module, together with account and leaderboard services. The user operates the simulated drone through the VR interface. The flight simulation combines control input with the drone and environment models, while the training and scenario manager configures the exercise and the telemetry and feedback module presents the resulting flight information. Account and leaderboard services support registration, saved performance results, local rankings, and optional global rankings through the online service.

S.2 Functionality

This section defines the externally observable functional and non-functional requirements of FPVR. Each requirement has a unique identifier and an individual rationale. Requirements apply to the initial release unless explicitly designated as an optional enhancement. Requirements refer to the *target headset*, the *reference PCVR computer*, the headset's *motion controllers*, and a *standard gamepad*; the specific devices are imposed technical choices recorded in P.2, so the requirements stay valid if the hardware changes. Numeric thresholds and scoring parameters in this section constitute a proposed, testable baseline to be approved by the project team and verified on target hardware.

S.2.1 Drone Flight Simulation

- **FR-FLT-01:** FPVR shall display a first-person view whose position and orientation follow the simulated drone during flight.
  Rationale: The view must correspond to the vehicle being controlled to provide an FPV training experience.
- **FR-FLT-02:** FPVR shall accept independently variable throttle, pitch, roll, and yaw inputs and produce a corresponding change in simulated drone behaviour for each input.
  Rationale: These are the four primary control axes expected in manual FPV flight.
- **FR-FLT-03:** When the drone is airborne and no upward thrust is applied, its vertical velocity shall decrease under simulated gravity.

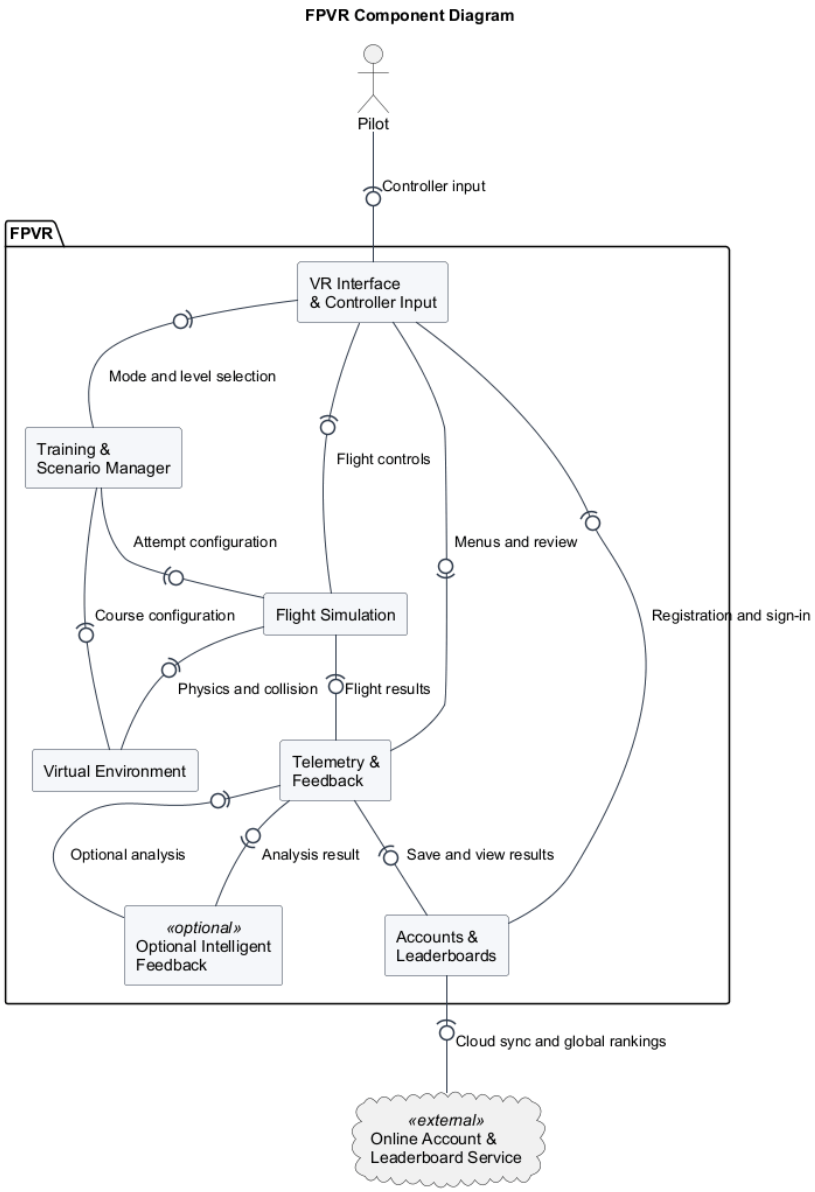


Figure 5: FPVR Logical Component Diagram

Rationale: Gravity is necessary to distinguish the flight model from unrestricted game-style movement and can be checked without prescribing a physics engine.

- **FR-FLT-04:** FPVR shall detect drone contact with course obstacles and ground surfaces during flight.
  Rationale: Contact detection is necessary to enforce obstacle challenges and initiate recovery.

- **FR-FLT-05:** Initial ground contact before the first takeoff of an attempt shall not count as a crash; contact with the ground after the drone has become airborne, or contact with a designated obstacle at any time, shall trigger a simulated crash.
  Rationale: This permits conventional takeoff while making the collision rule unambiguous. For acceptance testing, the airborne state begins when the drone is no longer in contact with its starting ground surface.

- **FR-FLT-06:** Each crash shall produce exactly one crash event, suspend normal flight input, and present the recovery behaviour applicable to the current mode.
  Rationale: A single event prevents a prolonged collision from generating duplicate penalties.

S.2.2 Training Modes and Level Management

- **FR-LVL-01:** FPVR shall provide free-flight practice without checkpoints, a completion timer, numerical scoring, or performance ratings.
  Rationale: Beginners need an unrestricted setting to learn the controls before attempting assessed challenges.

- **FR-LVL-02:** Following a crash in free-flight mode, the user shall be able to restore the drone to its free-flight starting position without restarting FPVR.
  Rationale: Quick recovery enables repeated practice without interruption.

- **FR-LVL-03:** FPVR shall provide at least three selectable obstacle-course levels, each with a distinct obstacle or checkpoint arrangement;

the levels shall be identified as introductory, intermediate, and advanced.
Rationale: Three differentiated configurations establish a verifiable minimum for varied practice without relying on subjective difficulty claims.

- **FR-LVL-04:** FPVR shall provide at least two visually distinct virtual environments, with at least one playable obstacle-course level in each.
Rationale: Distinct environments offer variation while permitting reuse of an environment by multiple levels.

- **FR-LVL-05:** Every obstacle-course level shall be selectable immediately without prerequisites or unlock conditions.
Rationale: Users with different skill levels should choose their own training difficulty.

- **FR-LVL-06:** The user shall be able to leave either training mode and return to the main menu without restarting the application.
Rationale: Users must be able to stop a VR activity without terminating the software.

- **FR-LVL-07:** Starting an obstacle-course attempt shall initialize checkpoint progress to zero, both timers to zero, crash count to zero, and the attempt's score state to its initial value.
Rationale: A fresh attempt must be independent of the previous attempt's performance.

S.2.3 Checkpoints and Crash Recovery

- **FR-CHK-01:** Each obstacle-course level shall have an ordered list of at least two checkpoints followed by a finish location.
Rationale: Ordered segments are needed for measurable progression and checkpoint-based practice.

- **FR-CHK-02:** FPVR shall record a checkpoint as completed only when the drone passes through the next checkpoint in the prescribed order.
Rationale: This makes completion dependent on following the intended course.

- **FR-CHK-03:** Passing through a later checkpoint before completing the next required checkpoint shall not advance progress.
  Rationale: This prevents bypassing the challenge sequence.

- **FR-CHK-04:** After a crash in an obstacle course, FPVR shall display the choices *Respawn* and *Restart Level*.
  Rationale: Users need both targeted practice and the option to begin a fresh scored attempt.

- **FR-CHK-05:** Respawn shall return the drone to the most recently completed checkpoint, or to the level start if none has been completed.
  Rationale: The player can retry the failed segment without losing earlier valid progress.

- **FR-CHK-06:** Respawn shall preserve completed checkpoints, accumulated total flight time, and the recorded crash count and score penalties.
  Rationale: Repeated failed attempts must continue to affect the overall performance result.

- **FR-CHK-07:** Restart Level shall return the drone to the level start and reset checkpoints, both timers, crash count, and score state.
  Rationale: Restarting is defined as a new attempt rather than a shortcut that retains previous penalties.

- **FR-CHK-08:** FPVR shall declare a level complete only after every checkpoint has been completed in order and the drone reaches the finish.
  Rationale: This gives a deterministic and testable definition of successful completion.

S.2.4 Timing and Performance Evaluation

- **FR-PER-01:** Each obstacle-course attempt shall have a total timer that measures accumulated active flight time from the start of the attempt to successful completion.
  Rationale: A single total time provides a comparable performance measure for a completed attempt.

- **FR-PER-02:** The total timer shall retain time spent in flight segments that end in crashes.
  Rationale: Failed attempts must not disappear from the overall completion-time result.

- **FR-PER-03:** Each obstacle-course attempt shall have a separate section timer measuring active flight time since the start of the current checkpoint segment.
  Rationale: Segment timing provides immediate feedback during repeated practice.

- **FR-PER-04:** The section timer shall reset to zero when the next required checkpoint is completed and when a checkpoint respawn occurs.
  Rationale: Each segment attempt should begin with a fresh local timer.

- **FR-PER-05:** Checkpoint respawn shall not reset the total timer; Restart Level shall reset both timers.
  Rationale: The two recovery choices intentionally have different effects on attempt history.

- **FR-PER-06:** Both timers shall pause while the crash-recovery menu is displayed and resume when active flight resumes; no menu time shall be added to either timer.
  Rationale: Decision time in a menu should not affect flight performance.

- **FR-PER-07:** FPVR shall increment the crash count once for each registered crash event in an obstacle-course attempt.
  Rationale: The count is required for meaningful penalties and a transparent review.

- **FR-PER-08:** At level completion, FPVR shall calculate a whole-number score as $\max(0, 1000 - \lfloor T \rfloor - 100C)$, where T is total active flight time in seconds and C is the number of registered crashes.
  Rationale: This proposed baseline is deterministic, easy to reproduce in tests, and penalizes both time and crashes. The constants are initial game-balance choices, not experimentally validated measures of pilot skill.

- **FR-PER-09:** FPVR shall assign the rating Gold for scores of at least 800, Silver for 600–799, Bronze for 400–599, and Needs Improvement for 0–399.
  Rationale: Fixed non-overlapping score bands make ratings unambiguous and verifiable; the bands are initial design choices rather than validated proficiency levels.

- **FR-PER-10:** The total timer shall stop when the level is successfully completed, and its final value shall be used in the score and post-flight review.
  Rationale: A fixed completion event ensures consistent reported results.

S.2.5 VR Interface and Controller Input

- **FR-UI-01:** FPVR shall present the flight view and training menus in the target headset.
  Rationale: Headset-based training is the core experience; the target headset is the agreed primary evaluation device (P.2).

- **FR-UI-02:** FPVR shall support both standalone execution on the target headset and PCVR execution with the target headset connected to the reference PCVR computer.
  Rationale: Both deployment paths are in the project scope; a single reference computer (P.2) makes PCVR testing repeatable.

- **FR-UI-03:** FPVR shall provide a flight-control configuration for the target headset's motion controllers in standalone mode and a standard gamepad in PCVR mode, each providing throttle, pitch, roll, and yaw.
  Rationale: This defines a minimum verifiable controller-to-execution-mode matrix without promising every controller in every mode.

- **FR-UI-04:** For each supported controller configuration, FPVR shall display a control guide identifying the assigned throttle, pitch, roll, and yaw inputs before the user begins flight.
  Rationale: Explicit instructions allow users to operate the simulator without assuming previous FPV experience.

- **FR-UI-05:** FPVR shall provide menu interactions for choosing training mode, selecting a level, starting flight, recovering from crashes, and

returning to the main menu.
Rationale: Every required gameplay transition must be accessible to the user.

- **FR-UI-06:** FPVR shall permit seated and standing operation without requiring the user to physically walk to control drone movement.
Rationale: Users should be able to operate the simulator within a stationary VR play area.

S.2.6 Player Accounts and Authentication

- **FR-ACC-01:** FPVR shall allow a user with network connectivity to register an account using a unique username and password.
Rationale: Persistent identity is needed to associate results with a player and support global ranking.

- **FR-ACC-02:** FPVR shall reject registration when the username is already registered or the password contains fewer than 12 characters, and shall display an explanatory message.
Rationale: Duplicate identity must be prevented, and a clear minimum password rule is easier to verify than an undefined strong-password requirement.

- **FR-ACC-03:** FPVR shall allow a registered user to sign in with valid credentials and shall reject invalid credentials without disclosing whether the username or password was incorrect.
Rationale: Authentication protects ownership of stored results while avoiding unnecessary account enumeration.

- **FR-ACC-04:** FPVR shall provide a sign-out action that ends the current authenticated session on that installation.
Rationale: Shared headsets or computers require a way to stop acting under another player's identity.

- **FR-ACC-05:** FPVR shall allow free-flight and obstacle-course practice without an authenticated account, but shall require authentication before submitting a result to the global leaderboard.
Rationale: Account or network failure should not block the core training experience, while public rankings need attributable submissions.

S.2.7 Result Storage and Leaderboards

- **FR-DAT-01:** After a completed obstacle-course attempt, FPVR shall save a local result record containing the level identifier, completion timestamp, total time, crash count, score, rating, and the active local player identifier.
  Rationale: A local record supports a device leaderboard and retains results when a network connection is unavailable.

- **FR-DAT-02:** For an authenticated player with network connectivity, FPVR shall submit the completed result to the online result service and associate it with that player's account.
  Rationale: Cloud association allows results to be retrieved across installations and included in global rankings.

- **FR-DAT-03:** FPVR shall retain a locally saved result when an online submission fails and shall allow the player to retry synchronization after connectivity is restored.
  Rationale: Temporary network failure should not erase a completed attempt.

- **FR-DAT-04:** FPVR shall display a local leaderboard for each obstacle-course level using results saved on the current installation.
  Rationale: Users can compare results without relying on an external service.

- **FR-DAT-05:** FPVR shall display a global leaderboard for each obstacle-course level using results successfully accepted by the online service.
  Rationale: Online rankings enable comparison between registered players across installations.

- **FR-DAT-06:** Both leaderboards shall rank results by descending numerical score, using shorter total completion time as the first tiebreaker and earlier completion timestamp as the second tiebreaker.
  Rationale: A complete ordering rule prevents ambiguous rankings for equal scores.

- **FR-DAT-07:** Each leaderboard shall show at least the top ten available results for the selected level, including rank, player display name, score, completion time, and crash count.

Rationale: A fixed minimum display capacity and field list make the leaderboard interface testable.

- **FR-DAT-08:** When the online leaderboard is unavailable, FPVR shall display an availability message and continue to provide the local leaderboard and training modes.
  Rationale: Online-service failure must not disable core gameplay.

- **FR-DAT-09:** A registered player shall be able to retrieve their previously synchronized completed results after signing in on another supported installation.
  Rationale: Cross-device retrieval is the user-visible benefit of cloud-backed result persistence.

- **FR-DAT-10:** FPVR shall distinguish results saved only locally from results successfully synchronized to the online service.
  Rationale: Users must not be misled into believing an offline result is already included in the global ranking.

S.2.8 Post-Flight Review

- **FR-REV-01:** FPVR shall display a post-flight review immediately after successful completion of an obstacle-course level.
  Rationale: Immediate feedback connects performance outcomes to the just-completed attempt.

- **FR-REV-02:** The review shall display the completed level, total completion time, completed checkpoints, crash count, numerical score, and rating.
  Rationale: These measures explain the result and support deliberate practice.

- **FR-REV-03:** From the review, the user shall be able to retry the same level, choose another level, or return to the main menu.
  Rationale: Users should be able to continue training without restarting the application.

- **FR-REV-04:** The standard review shall be displayed even when the network or intelligent feedback service is unavailable.

Rationale: Core performance feedback must not depend on optional external services.

- **FR-REV-05:** FPVR shall permit registered players to view their stored completed results from prior sessions, showing at least the fields specified in FR-DAT-01.
  Rationale: Account-linked result persistence is useful only if users can retrieve and inspect their earlier performance; this supersedes the earlier exclusion of historical results.

S.2.9 Intelligent Feedback

- **FR-AI-01:** FPVR shall generate a natural-language observation based on the completed attempt's available time, checkpoint, and crash data.
  Rationale: Feedback must be grounded in measurable attempt data rather than invented flight events.

- **FR-AI-02:** FPVR shall display it automatically in the post-flight review without requiring an additional user request.
  Rationale: Automatic presentation makes the enhancement accessible during the normal review workflow.

- **FR-AI-03:** Failure or unavailability of intelligent feedback shall not prevent display of the standard post-flight review.
  Rationale: An add-on feature must not become a single point of failure for required feedback.

S.2.10 Non-Functional Requirements

- **NFR-INST-01 (Installability):** FPVR shall install and launch as a standalone application on the target headset without a connected computer.
  Rationale: Standalone operation is a committed delivery mode, not merely an editor demonstration.

- **NFR-INST-02 (Installability):** FPVR shall launch in PCVR mode on the reference PCVR computer connected to the target headset using

a supported PCVR connection method.
Rationale: A specified reference computer makes the second deployment mode independently testable.

- **NFR-COMP-01 (Compatibility):** The target headset's motion controllers shall support the four flight axes in standalone mode, and a standard gamepad shall support the four flight axes in PCVR mode.
Rationale: This minimum matrix is testable without promising unsupported controller combinations.

- **NFR-PERF-01 (Rendering):** On the target headset in standalone mode, FPVR shall maintain an average of at least 72 rendered frames per second over a continuous 120-second normal-flight test in each of the two required environments.
Rationale: The 72-FPS threshold is an initial headset-oriented performance target; averaging over a fixed interval makes the result reproducible, although frame-time spikes should also be inspected during V&V.

- **NFR-PERF-02 (Input Response):** In the reference test environment, the elapsed time from a received flight-control input event to the first corresponding simulated state update shall not exceed 100 ms in at least 95% of 100 measured input events.
Rationale: This proposed software-response threshold makes responsiveness measurable without conflating it with headset display latency.

- **NFR-REL-01 (Reliability):** FPVR shall complete 20 consecutive crash-and-respawn cycles in each required obstacle-course environment without application termination or loss of checkpoint consistency.
Rationale: A repeatable stress test checks the recovery workflow that users will exercise frequently.

- **NFR-USAB-01 (Usability):** The control guide shall be accessible from the main menu and before the first flight of a newly launched session.
Rationale: Beginners need discoverable instructions without requiring prior experience.

- **NFR-SAFE-01 (VR Safety):** FPVR shall allow the user to pause or leave an active flight activity using an in-application menu without

removing the headset.
Rationale: Users must be able to stop an uncomfortable experience promptly.

- **NFR-SAFE-02 (Stationary Use):** All core flight, menu, and recovery actions shall be operable while seated or standing in one physical location.
  Rationale: Stationary operation reduces the need for movement within the user's physical surroundings.

- **NFR-SEC-01 (Credential Protection):** FPVR shall transmit account credentials only over encrypted connections and shall not store account passwords in plaintext on the device or online service.
  Rationale: Online account functionality introduces credential exposure risks absent from the original offline-only simulator.

- **NFR-SEC-02 (Access Control):** An unauthenticated user shall not be able to retrieve another player's non-public account data or submit a leaderboard result under another player's account identifier.
  Rationale: Public rankings must not compromise account privacy or permit identity spoofing.

- **NFR-DAT-01 (Persistence):** A result confirmed as synchronized shall remain retrievable after the player signs out, restarts FPVR, and signs in again.
  Rationale: Persistence across sessions is the defining acceptance condition for cloud-backed storage.

- **NFR-OFF-01 (Offline Availability):** Free-flight, obstacle-course play, crash recovery, local result saving, and standard post-flight review shall remain usable when the online service is unavailable.
  Rationale: Core training should remain available despite network failures; account registration, global ranking, and synchronization may require connectivity.

Baseline interpretation. The requirements above define the proposed initial acceptance baseline. The numerical score, rating bands, frame-rate target, input-response threshold, and reference PCVR platform are initial project specifications rather than empirically validated claims. Any later

change shall be recorded in revision history and propagated to verification tests, traceability links, and the affected scope sections.

Requirement priorities will be established in Section S.5, while verification and acceptance criteria will be defined in Section S.6.

S.3 Interfaces

IF-01: Entry and navigation: The VR and nunchuck controllers provide access to a main menu where users can select modes for free flight, available course choices, and return/exit actions. (FR-UI-04/FR-UI-05, FR-LVL-05/FR-LVL-06).

IF-02: Flight: Continuous throttle, pitch, roll, and yaw actions affect the drone's first-person view based on inputs from a controller (FR-FLT-02). The application will display progress during a course (FR-REV-02), where as free flight has no course timer, checkpoints, or score (FR-FLT-01/FR-FLT-02, FR-LVL-01).

IF-03: Retry: Upon failure to complete a course due to a crash or timeout, users can press a button on the controller to reset the drone to a prior checkpoint. Selection is an explicit menu action that can be interfaced with a controller. (FR-LVL-02, FR-CHK-04–FR-CHK-07;).

IF-04: Review: On course completion show the selected course, final total active time labelled in seconds, checkpoint results in course order, integer crash count, numerical score in points, and separate rating. Retry and activity selection remain available; prior-session navigation is absent. Unapproved scoring parameters must be identified as provisional in any prototype rather than presented as validated training results (FR-REV-01–FR-REV-05).

IF-05: Platform Interaction: Consumes pose/input availability and focus; produces frames and lifecycle responses through the supported runtime. The proposed unavailable-input presentation is descriptive and recoverable where the runtime permits (SR-02). There is no promise of public network APIs, desktop-only controls, or physical-drone interfaces.

IF-06: AI Feedback: The feedback provided by the AI system receives only separately approved current-attempt data and advises on possible places for improvement. The standard review is immediate and independent; an analysis may join it automatically if the optional capability is approved (FR-AI-01–FR-AI-03, SR-08). Output schemas, provider, timeout, and data permission are **TO BE REVIEWED** before external AI enters scope.

Interfaces describe visible information and action meanings. Exact widget layout, engine classes, storage structures, and input packages remain design choices. Proposed safety/error interactions are subject to their SR status.

S.4 Detailed usage scenarios

U-01: Start a Free-Flight Session

Primary Actor: FPVR user

Stakeholders of interest: Novice FPV pilots, drone and gaming enthusiasts

Precondition: The application is running with all necessary hardware, a supported headset and controller are available, and the main menu is visible from the VR.

Main success scenario:

- The pilot selects Free Flight from the main menu.
- The system loads the free-flight environment and places the drone at its starting position.
- The application initializes the controller inputs and displays the flight view.
- The pilot practises throttle, roll, pitch, and yaw in the open environment.
- The pilot takes off and manoeuvres the drone without course restrictions.
- The simulator provides no course timer, checkpoints, or score for the session.

- If the drone crashes, the pilot opens the recovery option and resets the drone.
- The system returns the drone to its grounded starting position and restores flight control.
- The pilot continues practising and may repeat the crash-and-reset cycle.
- The pilot selects Exit and the application returns to the main menu.

Secondary scenarios:

- If the pilot crashes repeatedly:
  1. The pilot selects Reset after each crash.
  2. The system counts the contact as a crash and opens the recovery state.
  3. The system clears the active motion and returns the drone to its grounded starting position.
  4. The pilot confirms that the controls are neutral and resumes practice.
  5. The pilot repeats these steps for each subsequent crash; no course score or completion result is created.
- If the controller disconnects:
  1. The application detects the loss of the controller connection.
  2. The system pauses flight input and prevents the last received axes from being applied continuously.
  3. The pilot reconnects or re-pairs the controller.
  4. The application reports that the controller is available and asks the pilot to place the controls in their neutral position.
  5. The pilot applies neutral controls and explicitly confirms Resume.
  6. The system restores flight control from the paused state.
- If an input axis is non-finite:
  1. The application detects a non-finite input value.
  2. The system rejects the value instead of using it for flight or scoring.

3. The application pauses the session when possible and asks the user to release and reapply the affected control.
4. The user restores valid controller input and explicitly resumes.

U-02: Running an Obstacle-Course Level

Primary Actor: FPVR user

Stakeholders of interest: Novice FPV pilots, drone and gaming enthusiasts

Precondition: An obstacle-course level with ordered checkpoints and a finish is available, and the user is at the level-selection interface.

Main success scenario:

- The user opens the level-selection interface.
- The application displays the available obstacle-course levels.
- The user selects an available course.
- The system loads the course and places the drone at the valid starting pose.
- The system initializes the attempt, checkpoint progress, collision count, and timer.
- The user takes off and flies toward the first required checkpoint.
- The user crosses each required checkpoint in the specified order.
- The system records each valid checkpoint crossing and advances the required checkpoint.
- The user reaches the finish after all required checkpoints are complete.
- The application validates the completed route, stops the timer, and opens the current-attempt review.

Secondary scenarios:

- If the user crosses a later checkpoint before the required checkpoint:
  1. The system detects the checkpoint trigger.

2. The system compares the triggered checkpoint with the next required checkpoint.
3. The system rejects the out-of-order crossing.
4. The application leaves the required checkpoint unchanged and informs the user to return to the route.
5. The user flies through the required checkpoint before continuing toward the later checkpoint.

- If the user crosses an already-completed checkpoint:
  1. The system detects the repeated checkpoint trigger.
  2. The system identifies the checkpoint as already recorded.
  3. The system ignores the repeated trigger and preserves the existing progress.
  4. The user continues from the current required checkpoint.
- If the controller disconnects:
  1. The application detects the loss of the controller connection.
  2. The system pauses flight input and prevents the last received axes from being applied continuously.
  3. The pilot reconnects or re-pairs the controller.
  4. The application reports that the controller is available and asks the pilot to place the controls in their neutral position.
  5. The pilot applies neutral controls and explicitly confirms Resume.
  6. The system restores flight control from the paused state.
- If a required checkpoint is missing:
  1. The application detects that the course does not contain the required checkpoint.
  2. The system marks the attempt invalid and prevents a finish from becoming a valid completion.
  3. The system pauses the session when possible and informs the user of the missing course data.
  4. The user exits the invalid attempt or retries after the course data is corrected.
- If an input axis is non-finite:

1. The application detects a non-finite input value.
2. The system rejects the value instead of using it for flight or scoring.
3. The application pauses the session when possible and asks the user to release and reapply the affected control.
4. The user restores valid controller input and explicitly resumes.

- If the drone position is out of bounds:
  1. The system detects that the drone is outside the valid course area.
  2. FPVR informs the user that they are outside and must return to the main course
  3. After a prolonged period, the system will reset the drone to the last valid position or checkpoint.

U-03: Recover from Crash during a Course.

Primary Actor: FPVR user

Stakeholders of interest: FPV pilots

Precondition: The user is attempting an obstacle course and is actively flying until they collide with an obstacle such as the ground, wall, or a goal.

Main success scenario:

- The user flies an active obstacle-course attempt.
- The drone collides with an obstacle, the ground, a wall, or a goal.
- The system detects the collision and stops active flight input.
- The system records one crash and displays the recovery menu.
- The user selects Restart.
- The application finds the most recently completed checkpoint, or the course start if no checkpoint has been completed.
- The system places the drone at the selected recovery position with a valid orientation.
- The system resets the section timer while preserving total time and crash penalties.

- The system restores the next required checkpoint and clears transient crash state.
- The user confirms neutral controls and resumes flight.

Secondary scenarios:

- If the drone makes repeated contact while the recovery menu is shown:
  1. The application detects the first collision and opens the recovery menu.
  2. The system marks the crash as handled and disables additional crash registration while the menu is displayed.
  3. The system ignores subsequent contact events during that recovery state.
  4. The user selects Restart or exits the attempt.
  5. The system records no more than one crash for that collision event.
- If the user selects Restart as a new attempt:
  1. The user chooses New Attempt rather than recovery at the checkpoint.
  2. The application asks the user to confirm the restart.
  3. The user confirms the restart.
  4. The system creates a new attempt at the course start.
  5. The system resets checkpoint progress, timers, collision count, and performance data.
  6. The user begins the new flight.
- If external networking or the optional AI service is unavailable:
  1. The application detects the unavailable service or failed connection.
  2. The system reports that the optional capability is unavailable.
  3. The application disables only the dependent optional feature.
  4. The user continues practising, recovering from crashes, and completing the course using standard local behavior.
  5. The system records and displays the standard result without waiting for the unavailable service.

- If the user declines consent for optional AI analysis:
  1. The application requests consent before sending the required flight data.
  2. The user selects Decline.
  3. The system records that optional analysis is unavailable for this attempt and does not send the data.
  4. The application continues the standard flight, recovery, and review flow.
  5. The system displays the standard result without changing its score.
- If the optional AI request times out:
  1. The application starts the optional request after consent is available.
  2. The system reaches the configured timeout without a valid response.
  3. The application cancels or abandons the optional request and reports the service as unavailable.
  4. The user continues to the standard result without waiting further.
  5. The system preserves the recorded score and rating.
- If the optional AI output is malformed:
  1. The application receives the optional response.
  2. The system validates the response before displaying or applying it.
  3. The application rejects malformed output and reports that AI feedback is unavailable.
  4. The system displays the standard review and preserves the recorded score, rating, and other results.

U-04: Replay and Analyze Flight Performance

Primary Actor: FPVR user

Stakeholders of interest: FPV pilots, drone training instructors, and the team responsible for the optional AI feedback service.

Precondition: The user has completed at least one course and the attempt has been recorded by the application. If AI feedback is enabled, the user has approved the data required for the optional analysis.

Main success scenario:

- The user opens the list of recorded flights.
- The application displays the locally stored recordings and their available results.
- The user selects a completed flight recording.
- The system loads the stored virtual telemetry data and footage.
- The application presents the recording in the VR or screen replay view.
- The user plays the recording and may pause, rewind, or fast-forward it.
- The system displays the standard performance results, including completion time, checkpoint results, collision count, numerical score, and performance rating.
- If optional AI feedback is enabled and consent is available, the application sends the permitted performance information for analysis.
- The system displays the resulting observations within the review without changing the standard results.
- The user exits the replay and the application returns to the main menu.

Secondary scenarios:

- If the selected recording is corrupted:
  1. The application attempts to read and validate the recording metadata and content.
  2. The system detects the corruption and stops loading the recording.
  3. The application displays an error explaining that the recording cannot be replayed.
  4. The user dismisses the error.

5. The application returns the user to the recording list or main menu without changing the stored result.

- If the selected recording is missing:
  1. The system looks up the selected recording and its referenced local files.
  2. The application detects that one or more required files are absent.
  3. The system displays an error and removes the unavailable item from the active replay view.
  4. The user dismisses the error and returns to the recording list or main menu.
- If the recording is incompatible with the current application version:
  1. The application reads the recording version metadata.
  2. The system compares it with the current supported recording format.
  3. The application prevents incompatible data from being interpreted as a replay.
  4. The system displays a warning and suggests updating the recording or the application.
  5. The user returns to the recording list or updates before trying again.
- If AI feedback is unavailable:
  1. The application checks whether the optional service is reachable and enabled.
  2. The system reports the unavailable capability.
  3. The application skips the optional request.
  4. The system displays the standard review and leaves the user in the replay flow.
- If the user declines AI consent:
  1. The application requests consent before sending permitted flight-performance data.
  2. The user selects Decline.

3. The system does not send the data for optional analysis.
4. The application displays the standard review without optional observations.

- If AI feedback times out:
  1. The application starts the optional analysis request.
  2. The system reaches the configured timeout without a usable response.
  3. The application stops waiting and marks the analysis unavailable.
  4. The system displays the standard review without interrupting replay controls.
- If the AI service fails to produce an analysis:
  1. The application receives an empty or failed analysis result.
  2. The system reports that no optional feedback is available.
  3. The application keeps the standard performance results visible.
  4. The user continues or exits the replay normally.
- If the AI response is malformed:
  1. The system validates the response before using it.
  2. The application discards the malformed response.
  3. The system records the optional analysis as unavailable.
  4. The application displays the standard review without changing the recorded score, rating, or other standard results.

U-05: Connecting to Multiplayer Training Session (optional)

Primary Actor: FPVR user

Stakeholders of interest: FPV pilots

Precondition: The user is running the application, has a stable internet connection, and has selected the optional multiplayer training mode. The application should be able to connect to an external database or server.

Main success scenario:

- The user selects Multiplayer Training from the main menu.

- The application verifies that the multiplayer option and network connection are available.
- The system requests a connection to an available training session.
- The application joins the session and receives its participant and course data.
- The system displays the participating users and selected training course.
- The application initializes the user's local flight state and performance data.
- The users complete the training session while the system synchronizes the session state.
- The system keeps each participant's flight state and performance data separate.
- The application detects the end of the session and validates the local result.
- The system displays the session results and returns the user to the main menu.

Secondary scenarios:

- If no training session is available:
  1. The application searches for an available training session.
  2. The system finds no session that can accept the user.
  3. The application informs the user that no training session is available.
  4. The user selects Retry to search again or Exit to return to the main menu.
  5. The system performs the selected action without creating a partial multiplayer result.
- If the connection is lost before the session starts:
  1. The application detects that the connection attempt failed.
  2. The system cancels the incomplete join operation.
  3. The application informs the user that the session could not be joined.

4. The user selects Retry or returns to the main menu.

- If the connection is lost during the session:
  1. The system detects the lost connection.
  2. The application pauses local multiplayer input and marks synchronization as invalid.
  3. The system attempts the supported reconnection behavior or waits for the session to end.
  4. If synchronization is restored, the application reports the restored state and the user explicitly resumes.
  5. If synchronization cannot be restored, the system ends the multiplayer session, informs the user, and does not fabricate a result.
  6. The user returns to the main menu or selects a single-user training mode.
- If the multiplayer service is unavailable:
  1. The application detects that the service cannot be reached.
  2. The system informs the user that Multiplayer Training is unavailable.
  3. The application leaves the multiplayer attempt unstarted.
  4. The user returns to the main menu and selects Free Flight or an obstacle-course level.
  5. The system starts the selected single-user training mode normally.

S.5 Prioritization

The order in which these requirements are phased in, with due dates, is given in Table 23 (P.3).

Table 12: Priority Assignment: Drone Flight Simulation

Requirement Link	Requirement Name	Priority	Rationale
FR-FLT-01	First-Person Flight View	High	Essential to the FPV experience; the view must follow the drone to support effective flight perception and control.
FR-FLT-02	Four-Axis Flight Controls	High	All four control axes are necessary for meaningful manual FPV flight training.
FR-FLT-03	Simulated Gravity	High	Fundamental to credible flight behaviour and realistic control practice.
FR-FLT-04	Collision Detection	High	Required to detect obstacle and ground contact and initiate crash recovery.
FR-FLT-05	Crash Trigger Rules	High	Establishes correct takeoff and crash behaviour, preventing false crashes and enforcing course challenges.
FR-FLT-06	Crash Event Handling	High	Prevents duplicate crash events and coordinates input suspension and recovery, protecting penalty accuracy.

Table 13: Priority Assignment: Training Modes and Level Management

Requirement Link	Requirement Name	Priority	Rationale
FR-LVL-01	Free-Flight Practice	High	Provides unrestricted practice for learning flight controls without assessment pressure.
FR-LVL-02	Free-Flight Recovery	High	Enables repeated practice after crashes without restarting the application.
FR-LVL-03	Three Differentiated Levels	High	Establishes the minimum required course content and introductory, intermediate, and advanced progression.
FR-LVL-04	Multiple Environments	Medium	Provides environmental variety and satisfies the required content scope, but is secondary to core flight functionality.
FR-LVL-05	Unrestricted Level Selection	Medium	Improves user choice and accessibility, although it is not fundamental to flight simulation.
FR-LVL-06	Return to Main Menu	High	Allows users to leave VR activities and navigate the application without terminating it.
FR-LVL-07	Attempt Initialization	High	Ensures attempts start with clean state, preventing previous progress, timings, and penalties from affecting results.

Table 14: Priority Assignment: Checkpoints and Crash Recovery

Requirement Link	Requirement Name	Priority	Rationale
FR-CHK-01	Ordered Checkpoints and Finish	High	Defines the course structure needed for progression, timing, and completion.
FR-CHK-02	Ordered Checkpoint Completion	High	Ensures progression depends on passing through checkpoints in the intended sequence.
FR-CHK-03	Prevention of Checkpoint Skipping	High	Preserves course integrity by preventing users from bypassing required segments.
FR-CHK-04	Crash Recovery Choices	High	Provides the essential options for resuming practice or starting a fresh attempt.
FR-CHK-05	Checkpoint Respawn	High	Allows users to retry a failed segment without repeating completed sections.
FR-CHK-06	Respawn State Preservation	High	Maintains valid progress and accumulated penalties during recovery.
FR-CHK-07	Level Restart and Reset	High	Provides a clean attempt state for consistent scoring and testing.
FR-CHK-08	Level Completion Validation	High	Defines deterministic completion required by scoring, storage, and review.

Table 15: Priority Assignment: Timing and Performance Evaluation

Requirement Link	Requirement Name	Priority	Rationale
FR-PER-01	Total Flight Timer	High	Provides the principal time measure for evaluating and comparing completed attempts.
FR-PER-02	Crash Time Retention	High	Prevents failed flight segments from being excluded from total completion time.
FR-PER-03	Section Timer	Medium	Provides segment-level feedback for targeted practice beyond the overall attempt time.
FR-PER-04	Section Timer Reset Rules	Medium	Ensures segment measurements remain meaningful after checkpoint completion and respawn.
FR-PER-05	Timer Recovery Rules	High	Maintains consistent timing across respawn and restart, directly affecting score accuracy.
FR-PER-06	Timer Pause During Recovery	High	Prevents menu decision time from distorting flight performance measurements.
FR-PER-07	Crash Counting	High	Supplies a required performance metric and the crash penalty used in scoring.
FR-PER-08	Numerical Score Calculation	High	Converts time and crash data into a reproducible score for ratings and rankings.
FR-PER-09	Performance Ratings	Medium	Makes results easier to interpret, although numerical scoring can operate without rating categories.
FR-PER-10	Final Timer and Score Integration	High	Ensures completed attempts use a fixed final time for scoring and review.

Table 16: Priority Assignment: VR Interface and Controller Input

Requirement Link	Requirement Name	Priority	Rationale
FR-UI-01	Target Headset VR Interface	High	Defines the primary target device for the intended headset-based training experience.
FR-UI-02	Standalone and PCVR Modes	High	Both deployment paths are committed project requirements.
FR-UI-03	Mode-Specific Controller Support	High	Provides the input mechanisms required for flight in both execution modes.
FR-UI-04	Controller Guidance	High	Identifies control assignments before flight, reducing confusion for beginners.
FR-UI-05	Menu Navigation and Actions	High	Makes all essential gameplay transitions accessible to users.
FR-UI-06	Seated and Standing Operation	High	Supports stationary use without requiring physical movement to control the drone.

Table 17: Priority Assignment: Player Accounts and Authentication

Requirement Link	Requirement Name	Priority	Rationale
FR-ACC-01	Account Registration	Medium	Enables attributable online results and global rankings, while core training remains available without an account.
FR-ACC-02	Registration Validation	High	Prevents duplicate usernames and enforces the specified minimum password length.
FR-ACC-03	Secure Sign-In	High	Protects account-linked results and avoids revealing which credential was incorrect.
FR-ACC-04	Sign-Out	Medium	Supports account separation on shared devices but is not required for basic training.
FR-ACC-05	Guest Training and Authenticated Submission	High	Preserves access to training while requiring authentication for global leaderboard submissions.

Table 18: Priority Assignment: Result Storage and Leaderboards

Requirement Link	Requirement Name	Priority	Rationale
FR-DAT-01	Local Result Storage	High	Preserves completed attempt data and provides the foundation for local rankings and offline continuity.
FR-DAT-02	Online Result Submission	Medium	Enables global rankings and cross-installation results but depends on connectivity and online infrastructure.
FR-DAT-03	Failed Submission Recovery	High	Prevents network failures from losing results and supports later synchronization.
FR-DAT-04	Local Leaderboard	Medium	Enables on-device performance comparison without depending on online services.
FR-DAT-05	Global Leaderboard	Medium	Supports cross-player comparison but requires online infrastructure and authenticated submissions.
FR-DAT-06	Leaderboard Ranking Rules	High	Defines deterministic ordering and tie-breaking, preventing ambiguous rankings.
FR-DAT-07	Leaderboard Display Requirements	Medium	Establishes a testable minimum display while depending on leaderboard functionality.
FR-DAT-08	Online Unavailability Handling	High	Ensures online service failures do not disable local rankings or core training.
FR-DAT-09	Cross-Installation Result Retrieval	Medium	Supports progress continuity across devices but depends on cloud persistence and authentication.
FR-DAT-10	Synchronization Status Display	High	Prevents users from mistaking locally saved results for results accepted by the online service.

Table 19: Priority Assignment: Post-Flight Review

Requirement Link	Requirement Name	Priority	Rationale
FR-REV-01	Automatic Post-Flight Review	High	Delivers immediate performance feedback after a successful attempt.
FR-REV-02	Review Metrics	High	Presents the measures needed to understand completion, score, and crash penalties.
FR-REV-03	Review Navigation	High	Allows users to continue training or leave without restarting the application.
FR-REV-04	Review Availability Without Network	High	Preserves essential performance feedback when optional online services are unavailable.
FR-REV-05	Historical Result Review	Medium	Enables progress tracking across sessions but is secondary to reviewing the current attempt.

Table 20: Priority Assignment: Intelligent Feedback

Requirement Link	Requirement Name	Priority	Rationale
FR-AI-01	Data-Grounded Feedback Generation	Low	Adds personalized interpretation of performance data but is not necessary for core training or standard review.
FR-AI-02	Automatic Feedback Presentation	Low	Improves convenience and discoverability of the optional intelligent feedback feature.
FR-AI-03	Feedback Failure Isolation	High	Prevents failure of the optional intelligent service from blocking the standard post-flight review.

Table 21: Priority Assignment: Non-Functional Requirements

Requirement Link	Requirement Name	Priority	Rationale
NFR-INST-01	Standalone Installability	High	Standalone delivery on the target headset is a committed deployment requirement.
NFR-INST-02	PCVR Installability	High	PCVR is a committed deployment mode and must be independently launchable on the reference platform.
NFR-COMP-01	Controller Compatibility	High	Correct controller support is necessary for flight in both required execution modes.
NFR-PERF-01	Rendering Performance	High	The specified frame-rate target supports smooth VR operation and helps reduce discomfort.
NFR-PERF-02	Input Response Time	High	Responsive control is fundamental to precise drone handling and must meet the specified threshold.
NFR-REL-01	Crash-Recovery Reliability	High	Repeated recovery cycles must preserve application stability and checkpoint consistency.
NFR-USAB-01	Control Guide Accessibility	High	Makes essential instructions discoverable to new users before flight.
NFR-SAFE-01	In-Application Pause and Exit	High	Allows users to stop or leave active VR activities promptly without removing the headset.
NFR-SAFE-02	Stationary Operation	High	Keeps core actions accessible from one physical location, reducing unnecessary movement in VR.
NFR-SEC-01	Credential Protection	High	Protects account credentials against interception and plaintext exposure.
NFR-SEC-02	Account Access Control	High	Prevents unauthorized access to private account data and fraudulent leaderboard submissions.
NFR-DAT-01	Synchronized Result Persistence	High	Ensures cloud-backed results remain retrievable across sign-outs and application restarts.
NFR-OFF-01	Offline Availability	High	Keeps core training and local functionality usable when online services fail.

S.6 Verification and acceptance criteria

This section states, for every requirement in S.2, how it will be **verified** (confirming FPVR was built correctly against the requirement) and how it will be **validated** (confirming the requirement meets the real training need). Each requirement identifier links to the requirement that defines it in S.2. Validation is by user evaluation sessions, supervisor/owner review, or expert review.

S.6.1 Drone Flight Simulation

- **FR-FLT-01**

  Verification: Move and rotate the drone in a test scene and confirm the headset view matches drone position and orientation, with no offset or lag between them.

  Validation: Users report the view feels like sitting in a real FPV drone.

- **FR-FLT-02**

  Verification: Apply each of throttle, pitch, roll, and yaw in isolation and in combination, and check that the drone's state responds in the correct axis and direction.

  Validation: Users with FPV experience confirm that the control response matches expectations.

- **FR-FLT-03**

  Verification: Run the zero-thrust drop fixture in PR-FLT-02 and compare the displacement with $gt^2/2$. Check that the drone falls when thrust is below hover level.

  Validation: Supervisor review of whether the flight feel is adequate for training.

- **FR-FLT-04**

  Verification: Fly into every obstacle type and ground surface in each environment and confirm contact is detected. Fly close to obstacles and confirm there are no false detections.

Validation: Users confirm that crashes occur where they visually expect them.

- **FR-FLT-05**

  Verification: Sit on the ground, then take off, and confirm no crash is registered. After takeoff, touch the ground and then an obstacle, and confirm a crash is registered each time.

  Validation: Users confirm that takeoff and landing feel natural, with no accidental crashes at takeoff.

- **FR-FLT-06**

  Verification: Trigger a crash and confirm exactly one crash event is recorded, flight input is blocked, and flight resumes only after a recovery action.

  Validation: Users confirm that the crash response is clear and fair.

S.6.2 Training Mode and Level Management

- **FR-LVL-01**

  Verification: Enter free flight and confirm no checkpoints, timer, or score are shown or computed.

  Validation: Novice users confirm free flight gives low-pressure practice.

- **FR-LVL-02**

  Verification: Crash repeatedly in free flight and confirm reset restores a flyable drone each time.

  Validation: Users confirm they can practise without frustration after a crash.

- **FR-LVL-03**

  Verification: Confirm at least three obstacle-course levels are selectable, labelled introductory, intermediate, and advanced, each with a distinct obstacle or checkpoint arrangement. Check that each loads and is completable.

Validation: Users rank the levels by difficulty, and the ranking is compared with the intended difficulty.

- **FR-LVL-04**

  Verification: Load each environment and confirm at least two visually distinct environments exist, each with at least one playable obstacle-course level.

  Validation: Users confirm that the environments look and feel different.

- **FR-LVL-05**

  Verification: On a fresh install, select every level in turn without completing any other level.

  Validation: Users confirm they can pick any level freely.

- **FR-LVL-06**

  Verification: Exit an active session to the main menu from free flight and from every level, and confirm the application does not restart or crash.

  Validation: Users confirm that exiting is easy to find and quick.

- **FR-LVL-07**

  Verification: Complete or abandon an attempt, start a new one, and inspect checkpoint progress, timers, collision count, and performance data for reset values.

  Validation: Supervisor review confirms that attempts are independent.

S.6.3 Checkpoint and Crash Recovery

- **FR-CHK-01**

  Verification: Inspect each course definition for an ordered sequence of at least two checkpoints followed by a finish location.

  Validation: Users confirm that the checkpoint order is understandable.

- **FR-CHK-02**

  Verification: Fly through checkpoints in order and confirm progress advances by exactly one each time.

  Validation: Users confirm that checkpoint progress matches what they experience.

- **FR-CHK-03**

  Verification: Fly through checkpoints out of order and confirm progress does not change.

  Validation: Users confirm that the course cannot be skipped and that guidance makes the next checkpoint clear.

- **FR-CHK-04**

  Verification: Crash during a course and confirm both the restart and respawn options are offered.

  Validation: Users confirm that the options are clear.

- **FR-CHK-05**

  Verification: Respawn after crashing before the first checkpoint, after checkpoint n, and at the last checkpoint. Check that the drone returns to the start or the last completed checkpoint.

  Validation: Users confirm that respawn lets them resume without confusion.

- **FR-CHK-06**

  Verification: Respawn and confirm completed checkpoints, total time, and collision penalties are all preserved.

  Validation: Supervisor review confirms respawn carries a fair penalty.

- **FR-CHK-07**

  Verification: Restart the level and confirm checkpoint progress, total time, section time, collision count, and score are all reset.

  Validation: Users confirm that restart gives a clean attempt.

- **FR-CHK-08**

  Verification: Reach the finish with all checkpoints completed in order, and again with a missed or out-of-order checkpoint. Only the first case may complete the level.

  Validation: Users confirm that completion feels earned and consistent.

S.6.4 Timing, Scoring, and Performance Evaluation

- **FR-PER-01**

  Verification: Run an attempt and compare the total timer with an independent stopwatch.

  Validation: Users confirm that the timer is visible and trustworthy.

- **FR-PER-02**

  Verification: Crash during a section and confirm that the time spent in that section is included in the total.

  Validation: Supervisor review confirms that failed sections should count.

- **FR-PER-03**

  Verification: Confirm the section timer starts from the beginning of each checkpoint segment and runs independently of the total timer.

  Validation: Users find per-section times useful for improving their flying.

- **FR-PER-04**

  Verification: Confirm the section timer resets on checkpoint completion and on respawn.

  Validation: Users confirm that the section timer reflects the current section.

- **FR-PER-05**

  Verification: Record the total time before and after respawn and confirm it does not drop.

Validation: Supervisor review confirms respawn keeps time consequences.

- **FR-PER-06**

  Verification: Open the crash-recovery menu, wait a known duration, and confirm both the total and section timers stay frozen and resume when flight continues.

  Validation: Users confirm that menu time does not affect performance.

- **FR-PER-07**

  Verification: Cause a known number of crashes and confirm the collision count increases by one per crash.

  Validation: Users confirm the collision count matches their experience.

- **FR-PER-08**

  Verification: For sample inputs, compare the score with a hand calculation of $\max(0, 1000 - \lfloor T \rfloor - 100C)$, for example $T = 95.7$, $C = 2$ gives 705. Confirm each added crash lowers the score by 100 and that a result that would be negative is shown as 0.

  Validation: Supervisor and user review of whether the score reflects flying quality.

- **FR-PER-09**

  Verification: Test the band boundaries, scores of 0, 399, 400, 599, 600, 799, 800, and 1000, and confirm they map to Needs Improvement, Bronze, Silver, and Gold as specified.

  Validation: User evaluation of whether the rating labels are meaningful.

- **FR-PER-10**

  Verification: Finish a course and confirm the displayed completion time equals the final total timer value and does not change afterwards.

  Validation: Users confirm the final time is trustworthy.

S.6.5 VR User Interface and Controller Input

- **FR-UI-01**

  Verification: Run the complete training experience, including the flight view and all menus, on the target headset.

  Validation: Users complete a session wearing the headset, and their feedback is recorded.

- **FR-UI-02**

  Verification: Run the same core scenarios standalone on the target headset and in PCVR mode with the target headset connected to the reference PCVR computer, and compare behaviour.

  Validation: Users confirm that both modes are usable.

- **FR-UI-03**

  Verification: Fly the same scenario with the target headset's motion controllers in standalone mode and with a standard gamepad in PCVR mode, and confirm throttle, pitch, roll, and yaw work in each.

  Validation: Users try both configurations and confirm either can be used to fly.

- **FR-UI-04**

  Verification: Confirm the control instructions list every flight input and its function, and that they appear before the first flight.

  Validation: New users fly their first flight after reading the instructions, and we observe whether they understand the controls.

- **FR-UI-05**

  Verification: Use the menus to select each training mode, select each level, and navigate back to the main menu.

  Validation: Users navigate the menus without help.

- **FR-UI-06**

  Verification: Complete all activities while seated and while standing in place, with no physical locomotion required.

  Validation: Seated and standing users confirm comfort and reachability.

S.6.6 Player Accounts and Authentication

- **FR-ACC-01**

  Verification: With network connectivity, register a new account with an unused username and a valid password, then confirm the account exists on the online service and can be used to sign in.

  Validation: New users register without help and report whether the process is clear.

- **FR-ACC-02**

  Verification: Attempt registration with an existing username, and with passwords of 11 and 12 characters. Confirm the first two are rejected with an explanatory message and the 12-character password is accepted.

  Validation: Users confirm the rejection messages tell them how to fix the problem.

- **FR-ACC-03**

  Verification: Sign in with valid credentials, then with a wrong password and with an unknown username. Confirm both failures show the same message and do not reveal which field was wrong.

  Validation: Supervisor review confirms the messages do not expose account information.

- **FR-ACC-04**

  Verification: Sign out, then confirm that leaderboard submission and stored-result access require signing in again on that installation.

  Validation: Users sharing a headset confirm they can hand it over without acting under another player's account.

- **FR-ACC-05**

  Verification: Without signing in, complete free flight and an obstacle course. Then attempt a global leaderboard submission and confirm it is refused until the user signs in.

  Validation: Users confirm they can practise without creating an account.

S.6.7 Result Storage and Leaderboards

- **FR-DAT-01**

  Verification: Complete an attempt and inspect the saved local record. Confirm it contains the level identifier, completion timestamp, total time, crash count, score, rating, and local player identifier, matching the post-flight review.

  Validation: Users confirm their saved results match what they achieved.

- **FR-DAT-02**

  Verification: While signed in and online, complete an attempt and confirm the result appears on the online service under that player's account.

  Validation: Users confirm their online results are correctly attributed to them.

- **FR-DAT-03**

  Verification: Disconnect the network, complete an attempt, and confirm the result is kept locally. Reconnect, retry synchronization, and confirm the result reaches the online service.

  Validation: Users confirm no completed attempt is lost after a network failure.

- **FR-DAT-04**

  Verification: Save results for two levels on one installation and confirm each level's local leaderboard shows only that level's results.

  Validation: Users confirm the local leaderboard is useful for comparing their own attempts.

- **FR-DAT-05**

  Verification: Submit results from two accounts and confirm each level's global leaderboard shows only results accepted by the online service.

  Validation: Users confirm the global leaderboard helps them compare against other players.

- **FR-DAT-06**

  Verification: Insert results with different scores, equal scores with different times, and equal scores and times with different timestamps. Confirm both leaderboards order them by score, then time, then timestamp.

  Validation: Users confirm the ranking order seems fair.

- **FR-DAT-07**

  Verification: With at least eleven results for a level, confirm each leaderboard shows the top ten with rank, display name, score, completion time, and crash count.

  Validation: Users confirm the shown fields are enough to understand the rankings.

- **FR-DAT-08**

  Verification: Make the online service unreachable, open the global leaderboard, and confirm an availability message appears while the local leaderboard and training modes still work.

  Validation: Users confirm the message explains why the global leaderboard is missing.

- **FR-DAT-09**

  Verification: Synchronize results on one installation, sign in to the same account on a second supported installation, and confirm the same results are retrieved.

  Validation: Users who switch devices confirm their history follows them.

- **FR-DAT-10**

  Verification: Create one result offline and one online, and confirm the display marks the first as local only and the second as synchronized.

  Validation: Users correctly identify which results are on the global leaderboard.

S.6.8 Post-Flight Review

- **FR-REV-01**

  Verification: Complete a level and confirm the review screen appears. Fail or exit a level and confirm the review does not appear.

  Validation: Users confirm that the review appears at a useful moment.

- **FR-REV-02**

  Verification: Confirm the review shows total completion time, checkpoint results, collision count, score, and rating, and that they match recorded values.

  Validation: Users confirm that the review content is sufficient for judging their performance.

- **FR-REV-03**

  Verification: From the review, retry the level and also choose another activity.

  Validation: Users confirm that the next step is obvious.

- **FR-REV-04**

  Verification: Disable or disconnect the intelligent feedback and confirm the standard review is still displayed in full.

  Validation: Supervisor review confirms the core review does not depend on the optional feature.

- **FR-REV-05**

  Verification: Sign in as a registered player with results from earlier sessions and confirm those results can be viewed, each showing at least the fields listed in FR-DAT-01.

  Validation: Users confirm the past results help them track their improvement.

S.6.9 Intelligent Feedback

These requirements are Low priority (S.5) and are verified in phase 6 of the phase-in plan (P.3).

- **FR-AI-01**

  Verification: Run attempts with known performance data and confirm the generated observations are consistent with the data.

  Validation: Expert and user review of whether the observations are accurate and useful.

- **FR-AI-02**

  Verification: Complete a level and confirm the feedback appears in the review without any user request.

  Validation: Users confirm the automatic presentation is acceptable.

- **FR-AI-03**

  Verification: Simulate a failure, timeout, or unavailability of the feedback and confirm the standard review still displays without interruption.

  Validation: Supervisor review confirms graceful degradation.

S.6.10 Non-Functional Requirements

- **NFR-INST-01**

  Verification: Install and launch FPVR on the target headset with no computer connected, starting from a clean device.

  Validation: An independent evaluator installs the application using only the documentation.

- **NFR-INST-02**

  Verification: Launch FPVR in PCVR mode on the reference PCVR computer connected to the target headset using a supported PCVR connection method.

  Validation: An independent evaluator launches it using only the documentation.

- **NFR-COMP-01**

Verification: Confirm the target headset's motion controllers control all four flight axes in standalone mode and a standard gamepad controls all four in PCVR mode, and record the tested combinations.

Validation: Users confirm that each controller is usable for flight.

- **NFR-PERF-01**

  Verification: On the target headset in standalone mode, fly normally for 120 continuous seconds in each of the two required environments and confirm the average frame rate is at least 72 FPS. Record frame-time spikes for review.

  Validation: Users report whether the visuals feel smooth and comfortable.

- **NFR-PERF-02**

  Verification: In the reference test environment, log the timestamp of 100 flight-control input events and of the first simulated state update for each. Confirm at least 95 of them are within 100 ms.

  Validation: Users report whether the drone responds to the controls without noticeable delay.

- **NFR-REL-01**

  Verification: Run 20 consecutive crash-and-respawn cycles in each required obstacle-course environment and confirm the application does not terminate and checkpoint progress stays consistent.

  Validation: Users practise extended sessions and report any instability.

- **NFR-USAB-01**

  Verification: Confirm the control guide is reachable from the main menu and is shown before the first flight of a newly launched session.

  Validation: First-time users are timed and observed to see whether they understand the controls.

- **NFR-SAFE-01**

  Verification: While in each activity, leave using the controller only, without taking off the headset.

Validation: Users confirm they can stop and exit quickly when uncomfortable.

- **NFR-SAFE-02**

  Verification: Perform every core flight, menu, and recovery action while seated and while standing in one marked spot, and confirm none requires leaving it.

  Validation: Users confirm they never feel the need to move around the play area.

- **NFR-SEC-01**

  Verification: Capture network traffic during registration and sign-in and confirm credentials are only sent over encrypted connections. Inspect device storage and the online service's account records and confirm no password is stored in plaintext.

  Validation: Supervisor review of the credential handling.

- **NFR-SEC-02**

  Verification: Without signing in, request another player's non-public account data and submit a result using another player's account identifier. Confirm both are rejected.

  Validation: Supervisor review confirms leaderboard results cannot be spoofed.

- **NFR-DAT-01**

  Verification: Synchronize a result, sign out, restart FPVR, sign in again, and confirm the result is still retrievable.

  Validation: Users confirm their synchronized results are never lost.

- **NFR-OFF-01**

  Verification: Disable the network or make the online service unavailable, then run free flight, an obstacle course, crash recovery, local result saving, and the standard review. Confirm each works.

  Validation: Users confirm they can train normally without a connection.

S.6.11 Proposed Quantitative and Interaction Specifications

All of these remain PROPOSED. Their verification is defined now so the cases are ready if they are approved. Earlier frame-rate, input-response, score, and rating proposals are now specified directly by NFR-PERF-01, NFR-PERF-02, FR-PER-08, and FR-PER-09, and are verified there.

- **PR-FLT-01**

  Verification: Hover near the clearance threshold of `TAKEOFF_CLEARANCE_M` and confirm airborne latches when it is first reached and does not clear when returning below it. Check respawn and restart initialize it correctly.

  Validation: Users confirm that takeoff feels natural. The clearance is checked against the drone size and course scale.

- **PR-FLT-02**

  Verification: Run the zero-thrust, zero-drag, zero-velocity fixture for `GRAVITY_TEST_SECONDS` and compare the displacement with $gt^2/2$ using `REFERENCE_GRAVITY` within `GRAVITY_REL_ERROR`. Document the fixture's initial pose and velocity.

  Validation: Supervisor review of the reference drone and realism tolerances, which remain TO BE REVIEWED.

- **PR-CTL-01**

  Verification: For each approved controller and mode combination, confirm each proposed controller action produces the stated action, and that unsupported or reserved buttons are not advertised.

  Validation: Users with each controller confirm that the mapping is learnable.

S.6.12 Proposed Safety, Security, and Failure Behaviour

All of these remain PROPOSED. Their verification is defined now so the cases are ready if they are approved.

- **SR-01**

  Verification: Confirm the safety, age, space, posture, and discomfort information appears before the first flight and that no unsupported medical or room claims are made.

  Validation: Expert review of the wording and user comprehension checks.

- **SR-02**

  Verification: Simulate loss of tracking, input, focus, and controller connection. Confirm that flight and both timers stop, stale controls are discarded, and resume requires available input, neutral controls, and explicit confirmation.

  Validation: Users experience the pause and report it as clear and safe.

- **SR-03**

  Verification: In seated and standing modes, reach pause, recenter, and exit. Confirm that flight and timers freeze and that no menu requires walking or reaching the floor.

  Validation: Seated and standing users evaluate comfort and reachability.

- **SR-04**

  Verification: Induce sustained frame-rate drops. Confirm that after `PERFORMANCE_FAILURE_SECONDS` below target over the `PERFORMANCE_WINDOW_SECONDS` window, FPVR pauses, explains why, and resumes only with confirmation.

  Validation: Comfort effectiveness is PENDING VERIFICATION through user trials.

- **SR-05**

  Verification: Inject non-finite or out-of-range inputs, invalid checkpoint orders, and intersecting recovery poses. Confirm invalid configuration is rejected before flight, runtime faults enter a recoverable error state, and progress, scores, and stale controls are unaffected.

  Validation: Supervisor review of the failure behaviour.

- **SR-06**

  Verification: With no account signed in, run core training, scoring, and review and inspect network traffic to confirm no attempt data is sent. Then sign in and confirm only the result fields in FR-DAT-01 are sent to the online service.

  Validation: Owner review confirms that only the data needed for online results leaves the device.

- **SR-07**

  Verification: Inspect the storage used by the application after several attempts. Confirm that only the result records in FR-DAT-01 are kept, and that other attempt data is discarded when a new attempt starts or the review is left.

  Validation: Owner review of retention scope.

- **SR-08**

  Verification: If external AI is proposed, review provider, privacy, retention, and opt-in. Confirm that feedback is labelled advisory, cannot change checkpoint, timer, crash, score, or rating state, and that timeout, error, or declined consent leaves the standard review intact.

  Validation: Supervisor and owner approval before any transfer is enabled.

(P) Project

P.1 Roles and personnel

- **Team Lead and Flight Physics Developer (Ibrahim Tariq)** – Tracks deadlines and keeps the project on schedule. Implements the flight model, gravity, and VR camera, and owns frame-rate performance.
- **Team Liaison and Hardware Integration Developer (John Cho)** – Main point of contact with the supervisor, teaching assistant, and instructor, and shares their updates with the team. Implements controller input and headset integration, and owns compatibility testing across headset and controller pairs.
- **Training Environments Lead (Sankalpa Chhetri Dhakal)** – Builds the obstacle courses and virtual environments. Leads the investigation of the optional ML (machine learning)/AI flight feedback.
- **Game State Developer (Vikram Chandar)** – Implements game state, checkpoints, crash recovery, and timing, and keeps course progress consistent across crashes and restarts.
- **Scoring and Player Data Developer (Arjun Sasidaran)** – Implements scoring, ratings, flight data recording, and the post-flight review, along with local and online result storage, leaderboards, and player accounts.
- **Supervisor (Dr. Douglas Down)** – Provides guidance on project scope, the reference drone and realism goals, and milestone planning. The team meets with the supervisor every two weeks.
- **Teaching Assistant** – Answers course-related questions and gives feedback on deliverables, reached through the team liaison.

In addition, every member takes turns as meeting chair, notetaker, and quality assurance (QA) reviewer; QA for a deliverable is never its main author, and every pull request is reviewed by a member other than its author.

P.2 Imposed technical choices

- **Game engine** – Unity, imposed by the confirmed project scope for its VR, input, and physics support.
- **Programming language** – C#, the Unity scripting language selected in the Development Plan.
- **VR headset** – Meta Quest 3S. This is the *target headset* in S.2, run both standalone and through PCVR (FR-UI-01, FR-UI-02). It was chosen as the agreed primary evaluation device.
- **Controllers** – The Quest 3S *motion controllers* in standalone mode and an Xbox-compatible gamepad, the *standard gamepad* in S.2, in PCVR mode (FR-UI-03).
- **PCVR platform** – A Windows 11 computer, the *reference PCVR computer* in S.2 (NFR-INST-02). A single reference operating system keeps PCVR testing repeatable.
- **XR integration** – Unity's OpenXR Plugin and XR Interaction Toolkit; exact package versions are an open decision (P.7).
- **Online service** – Accounts, result synchronization, and the global leaderboard (FR-ACC, FR-DAT) require an online service with authentication and data storage; the provider is an open decision (P.7).
- **Version control and review** – Git and GitHub, with one branch per issue and pull-request review before merging.
- **Continuous integration** – GitHub Actions, which builds the project documentation on every pull request.
- **Not required for the core system** – Multiplayer networking and custom RF (radio-frequency) hardware are not needed to meet the requirements in S.2. Multiplayer and dedicated FPV transmitter support are stretch goals (LC-08); the technology each would need is listed in P.5.

P.3 Schedule and milestones

Table 22 lists the course milestones and their due dates from the capstone calendar.

Table 22: Project milestones and due dates

Milestone	Due date
Problem Statement, PoC Plan, Development Plan	Complete (Sept. 2026)
SRS and Hazard Analysis	Oct. 19, 2026
SRS peer review issues	Oct. 22, 2026
VnV Plan	Nov. 2, 2026
VnV Plan peer review issues	Nov. 5, 2026
Design Document Rev −1 and CD-Fall	Nov. 16, 2026
Design Document Rev −1 peer review	Nov. 19, 2026
Proof of Concept (PoC) Demonstration	Nov. 23 – Dec. 3, 2026
Design Documentation	Jan. 18, 2027
Module Implementation	Jan. 21, 2027
Design Documentation peer review	Jan. 25, 2027
Revision 0 Demonstration	Feb. 1 – Feb. 11, 2027
V&V Report and CD-Winter	Mar. 8, 2027
V&V Report and CD-Winter peer review	Mar. 11, 2027
Final Demonstration (Revision 1)	Mar. 15 – Mar. 21, 2027
Final Documentation (Revision 1)	Mar. 31, 2027
Capstone EXPO	Apr. 7, 2027

Table 23 gives the order in which requirements are phased in. Each phase builds on the ones before it: the offline training flow that every later phase depends on comes first, the online account and result features come after it, and the Low-priority intelligent feedback (FR-AI) comes last. Each requirement's priority is given in S.5; within a phase, Medium-priority requirements are the first to be deferred if the phase runs late. Each phase is due at a milestone in Table 22; phases 1 and 2 form the proof of concept, which the Development Plan defines as VR support, a simple flight model, and controller input. Phase 5 does not depend on the flight work, so it is developed alongside phases 3 and 4 from January and is due with the V&V Report. A phase is complete when its requirements are implemented and pass the verification cases specified in the VnV Plan.

Table 23: Requirement phase-in plan

Phase	Requirements	Due date / milestone
1. Deployment, input, and safety	FR-UI-01 to FR-UI-04, FR-UI-06, NFR-INST-01, NFR-INST-02, NFR-COMP-01, NFR-SAFE-01, NFR-SAFE-02	Nov. 23 – Dec. 3, 2026 (PoC Demonstration)
2. Free flight	FR-FLT-01 to FR-FLT-06, FR-LVL-01, FR-LVL-02, NFR-PERF-02	Nov. 23 – Dec. 3, 2026 (PoC Demonstration)
3. Ordered courses, recovery, and timing	FR-LVL-03 to FR-LVL-07, FR-CHK-01 to FR-CHK-08, FR-PER-01 to FR-PER-07, FR-UI-05, NFR-PERF-01, NFR-REL-01	Jan. 21, 2027 (Module Implementation)
4. Scoring, review, and local results	FR-PER-08 to FR-PER-10, FR-REV-01 to FR-REV-04, FR-DAT-01, FR-DAT-04, FR-DAT-06, FR-DAT-07, NFR-USAB-01, NFR-OFF-01	Feb. 1 – Feb. 11, 2027 (Revision 0 Demonstration)
5. Accounts and online results	FR-ACC-01 to FR-ACC-05, FR-DAT-02, FR-DAT-03, FR-DAT-05, FR-DAT-08 to FR-DAT-10, FR-REV-05, NFR-SEC-01, NFR-SEC-02, NFR-DAT-01	Mar. 8, 2027 (V&V Report)
6. Intelligent feedback	FR-AI-01 to FR-AI-03	Mar. 15 – Mar. 21, 2027 (Final Demonstration); verified in the Revision 1 V&V Report; dropped first if the schedule slips

P.4 Tasks and deliverables

Table 24 lists the tasks that produce each deliverable, with the milestone dates from Table 22. The team lead tracks the dates and the area leads in P.1 own the tasks in their areas.

Table 24: Tasks and deliverables

Task	Description / steps	Deliverable	Due date
Requirements and hazards	Write the Goals, Environment, System, and Project books; give every requirement a unique ID and rationale; identify hazards with an FMEA and derive safety and security requirements; trace each requirement to its goal, hazards, and verification.	SRS and Hazard Analysis	Oct. 19, 2026
SRS peer review	Review the assigned team's SRS against the rubric and raise more than five substantive GitHub issues; triage the issues raised on this SRS and assign each to an owner.	SRS peer review issues	Oct. 22, 2026
Verification planning	Write a test case for every requirement in S.2, with inputs, expected outputs, and pass criteria; write step-by-step procedures for phases 1 and 2; define how frame rate and input response are measured on the headset (NFR-PERF-01, NFR-PERF-02).	VnV Plan	Nov. 2, 2026
VnV peer review	Review the assigned team's VnV Plan and raise issues; address the issues raised on this plan.	VnV Plan peer review issues	Nov. 5, 2026

Task	Description / steps	Deliverable	Due date
Architecture	Decompose the system into modules (input, flight model, course and checkpoint state, timing and scoring, review, result storage and leaderboards, accounts, user interface); define each module's secrets and interfaces; trace every High-priority requirement to a module.	Design Document Rev −1 and CD-Fall	Nov. 16, 2026
Design peer review	Review the assigned team's design document and raise issues; address the issues raised on this design.	Design Document Rev −1 peer review	Nov. 19, 2026
VR and controller integration (phase 1)	Set up the Unity project with OpenXR; build and launch on the Quest 3S in standalone and PCVR modes; map throttle, pitch, roll, and yaw for the Quest 3S motion controllers and the gamepad (FR-UI-03); add the control guide, main menu, pause menu, and exit (FR-UI-04, NFR-SAFE-01).	PoC Demonstration	Nov. 23 – Dec. 3, 2026
Flight model (phase 2)	Implement four-axis flight with gravity, takeoff, and landing (FR-FLT); add the first-person camera; measure input response (NFR-PERF-02) and profile frame rate on the headset; tune controller dead zones.	PoC Demonstration	Nov. 23 – Dec. 3, 2026
Detailed design	Specify each module's interface, state variables, and behaviour, including the checkpoint, crash, and timer state transitions and the local and online result states.	Design Documentation	Jan. 18, 2027

Task	Description / steps	Deliverable	Due date
Courses, recovery, and timing (phase 3)	Build at least three obstacle courses in at least two environments (FR-LVL); place ordered checkpoints; detect crashes, respawn at the last checkpoint, and restart a course (FR-CHK); add the total and section timers and crash count (FR-PER-01 to FR-PER-07) and the level-select and crash-recovery menus (FR-UI-05); confirm 72 FPS in both environments (NFR-PERF-01); run 20 consecutive crash-and-respawn cycles in each environment (NFR-REL-01).	Module Implementation	Jan. 21, 2027
Scoring, review, and local results (phase 4)	Implement the score and rating (FR-PER-08 to FR-PER-10); build the post-flight review screen (FR-REV); save each completed attempt locally and show the local leaderboard (FR-DAT-01, FR-DAT-04); confirm the core flow works with no network (NFR-OFF-01); integrate with phases 1–3 and rehearse the demonstration.	Revision 0 Demonstration	Feb. 1 – Feb. 11, 2027
Accounts and online results (phase 5)	Set up the online service; implement registration, sign-in, and sign-out (FR-ACC); submit and synchronize results, show the global leaderboard, and let players view past results (FR-DAT, FR-REV-05); test credential protection and access control (NFR-SEC-01, NFR-SEC-02) and behaviour when the service is unavailable (FR-DAT-08).	V&V Report and CD-Winter	Mar. 8, 2027

Task	Description / steps	Deliverable	Due date
Verification	Run every test case from the VnV Plan on the declared hardware; record the configuration and observed results; log each failure as a GitHub issue and retest after the fix.	V&V Report and CD-Winter	Mar. 8, 2027
Intelligent feedback (phase 6) and polish	Evaluate a pre-trained model on recorded attempt data and add feedback to the post-flight review (FR-AI); if it is not ready, fix defects from verification and improve comfort and usability instead.	Final Demonstration (Revision 1)	Mar. 15 – Mar. 21, 2027
Final documentation	Update the SRS, Hazard Analysis, VnV Plan and Report, and design documents to match the delivered system; write the user guide and installation instructions; complete the reflections.	Final Documentation (Revision 1)	Mar. 31, 2027
EXPO	Prepare the poster and a safe, supervised headset demonstration space; plan a short demonstration course for visitors.	Capstone EXPO	Apr. 7, 2027

A requirement counts as met only when its test case has been run on supported hardware and the configuration and observed results are recorded.

P.5 Required technology elements

- **VR Headset** – A Meta Quest 3S headset for both standalone and PCVR play.
- **Controllers** – Quest 3S motion controllers and an Xbox-compatible gamepad.

- **PCVR Host** – A Windows 11 computer and a supported PCVR connection to the headset.
- **Game Engine and Language** – Unity with C#, including its physics, input, and rendering systems.
- **XR Packages** – OpenXR Plugin and XR Interaction Toolkit for headset and controller integration.
- **Profiling Tools** – Unity Profiler and headset performance tools to measure frame rate, input response, and state transitions.
- **Local Storage** – On-device storage for saved results and the local leaderboard (FR-DAT-01, FR-DAT-04).
- **Online Service** – A hosted service providing account authentication, a result database, and the global leaderboard (FR-ACC, FR-DAT); the provider is an open decision (P.7).
- **Version Control** – GitHub for source code, issues, and pull requests.
- **Continuous Integration** – GitHub Actions with a LaTeX build for the project documents.
- **Test Space** – A safe, clear, stationary area for headset testing.
- **AI Model** – A pre-trained model, such as Qwen2-VL, evaluated for the intelligent feedback in phase 6 (FR-AI).
- **Network Stack (Stretch Goal)** – Unity Netcode, Unity Transport, and the Unity Multiplayer SDK, as identified in the Development Plan, needed only if multiplayer is taken on (LC-08).
- **RF Hardware (Stretch Goal)** – A dedicated FPV radio transmitter and a receiver module using an open-source radio protocol, needed only if transmitter support is taken on (LC-08); the protocol and hardware are an open decision (P.7).

Exact versions, PC specifications, and profiling equipment are open decisions listed in P.7.

P.6 Risk and mitigation analysis

- **Risk: VR or Controller Incompatibility**
  - Description: The Quest 3S, a controller, or an execution mode may not launch or accept input.
  - Mitigation: Integrate the headset and controllers first (phase 1), test both controller and mode pairs early, and record any problems (NFR-COMP-01). Responsible: John Cho.
- **Risk: VR Performance Shortfall**
  - Description: A representative course may fall below the target frame rate or exceed the input-response threshold, causing discomfort.
  - Mitigation: Profile each phase on the headset, and cut optional content before core features (NFR-PERF-01, NFR-PERF-02). Responsible: Ibrahim Tariq.
- **Risk: Unrealistic Flight Physics**
  - Description: The flight model may not feel like a real FPV drone because no reference drone has been chosen.
  - Mitigation: Agree on a reference drone with the supervisor and build the flight model incrementally from gravity checks (FR-FLT). Responsible: Ibrahim Tariq.
- **Risk: Inconsistent Game State**
  - Description: Crashes, checkpoints, and timers may disagree, giving wrong scores.
  - Mitigation: Define the checkpoint, crash, and timer state changes in the detailed design and test each one directly (FR-CHK, FR-PER). Responsible: Vikram Chandar and Arjun Sasidaran.
- **Risk: Online Service Delays or Security Flaws**
  - Description: The online service may take longer to set up than planned, be unavailable, or expose account credentials.

  - Mitigation: Build local results first (phase 4) so the core flow never depends on the network (NFR-OFF-01); use an established authentication service instead of writing one; test credential handling and access control (NFR-SEC-01, NFR-SEC-02); if phase 5 runs late, defer its Medium-priority requirements and keep the local leaderboard. Responsible: Arjun Sasidaran.

- **Risk: User Discomfort or Unsafe Play Space**
  - Description: Motion sickness, collisions with real objects, or hard to reach menus may harm testers.
  - Mitigation: Review the safety requirements in the Hazard Analysis and platform guidance before any user trials, keep an exit available without removing the headset (NFR-SAFE-01), keep all actions usable from one spot (NFR-SAFE-02), and stop testing when a user cannot continue safely. Responsible: rotating QA role.

- **Risk: AI Feedback Scope Creep**
  - Description: The AI feedback feature may take time away from core features.
  - Mitigation: Keep AI feedback in phase 6 and only start it after the Revision 0 demonstration (FR-AI). Responsible: Sankalpa Chhetri Dhakal.

- **Risk: Unrealistic Schedule or Workload**
  - Description: Tasks may take longer than planned or workload may become unbalanced.
  - Mitigation: Track all tasks on the GitHub Kanban board, review progress at each weekly meeting, reassign work as needed, and defer Medium- and Low-priority requirements before High ones (S.5). Responsible: Ibrahim Tariq.

Hazards to users are analysed separately in the Hazard Analysis.

P.7 Requirements process and report

- **Initial Elicitation:** Requirements were gathered from the problem statement, the Development Plan, meetings with the supervisor, competing simulators, and Meta Quest documentation. Sources are listed in G.7.
- **Elicitation Plan:**
  - Meetings: Weekly team meetings, in person or on Discord, and supervisor meetings every two weeks; each has an agenda and notes in a GitHub issue.
  - Communication: Discord for day-to-day discussion; the team liaison handles communication with the supervisor, TA, and instructor.
  - Documentation: Tasks, decisions, and feedback are tracked as GitHub Issues, and every change is made through a reviewed pull request.
  - Process Update: This section is updated as the project progresses to record lessons learned, review feedback, and revisions. Each requirement change records its source, rationale, affected IDs, and date.
- **Open Decisions:** These must be resolved before the affected requirements are accepted.
  - Hardware: PC specifications and Unity and package versions (NFR-INST-02, NFR-COMP-01).
  - Performance: confirm the 72 FPS and 100 ms thresholds on the Quest 3S and agree how they are measured (NFR-PERF-01, NFR-PERF-02).
  - Flight model: reference drone, accuracy targets, and controller dead zones (FR-FLT).
  - Scoring: calibration of the score constants and rating bands (FR-PER-08, FR-PER-09).
  - Online service: which provider to use for accounts, result synchronization, and the global leaderboard, and how credentials are stored (FR-ACC, FR-DAT, NFR-SEC-01).

- Safety and comfort: the safety requirements from the Hazard Analysis, comfort options, and procedures for testing with users (NFR-SAFE-01, NFR-SAFE-02).
- AI feedback: which model to use, whether it runs on the device or as a service, and how attempt data is handled (FR-AI).
- Stretch goals: which networking package to use for multiplayer and which radio protocol and hardware to use for FPV transmitters (LC-08).

Likely changes

Each likely change lists the requirements it affects and how they could change. These are variation points to isolate in later design; no module architecture is prescribed here.

LC-01: Device support. FR-UI-01, FR-UI-02, FR-UI-03, NFR-INST-01, NFR-INST-02, NFR-COMP-01. Other Quest models, such as the Quest 3, or extra controller and mode pairs, such as a gamepad in standalone mode, may be added once the Quest 3S baseline works, and the reference PCVR computer may change.

LC-02: Flight tuning. FR-FLT-02, FR-FLT-03, FR-FLT-05. Gravity, thrust, control response, and the takeoff rule may be retuned once a reference drone is agreed with the supervisor.

LC-03: Scoring and ratings. FR-PER-08, FR-PER-09, FR-DAT-06. The score constants (1000 points, 100 points per crash) and rating bands are initial choices and may be recalibrated after playtesting; leaderboard order follows whatever score is used.

LC-04: Performance thresholds. NFR-PERF-01, NFR-PERF-02. The 72 FPS and 100 ms targets are proposed values and may be raised or lowered after they are measured on the Quest 3S.

LC-05: Content and comfort. FR-LVL-03, FR-LVL-04, FR-UI-06, NFR-SAFE-01, NFR-SAFE-02. More levels, environments, and comfort or accessibility options may be added; the minimum of three levels in two environments stays fixed.

LC-06: Online services. FR-ACC-01 to FR-ACC-05, FR-DAT-02, FR-DAT-05, FR-DAT-09, FR-REV-05, NFR-SEC-01, NFR-DAT-01. The provider, password rule, and leaderboard details may change, and if the online service is not ready the Medium-priority online features may be deferred, leaving local results only.

LC-07: Intelligent feedback. FR-AI-01 to FR-AI-03. The model, the data it uses, and whether it runs on the device or as a service may change, and as the only Low-priority feature it may be deferred.

LC-08: Stretch goals. FR-UI-03, NFR-COMP-01. Multiplayer and dedicated FPV transmitter support may be added after the required features are complete. Each would add new requirements, bring in a networking package or radio hardware (P.5), and change the limits in G.6.

Unlikely changes

Each unlikely change lists the requirements it protects and why they are not expected to change.

UC-01: Simulation only. G.6. FPVR will not control physical drones or provide certification, because that would change the project's purpose and bring in safety and regulatory responsibilities the team cannot take on.

UC-02: Four-axis control. FR-FLT-02, FR-UI-03, FR-UI-04, NFR-COMP-01. Throttle, pitch, roll, and yaw are how every real FPV drone is flown, so changing them would break the FPV training goal.

UC-03: Recovery and scoring rules. FR-CHK-02, FR-CHK-05 to FR-CHK-08, FR-PER-05, FR-PER-07. Ordered checkpoints, keeping penalties on Respawn, and a fresh attempt on Restart Level are what make results comparable; changing them would make stored scores and leaderboards (FR-DAT-06) meaningless.

UC-04: Headset-based training. FR-UI-01, FR-UI-06, NFR-INST-01. Immersive VR is what sets FPVR apart from desktop simulators (G.2), and desktop-only play is excluded in G.6.

UC-05: Offline core training. NFR-OFF-01, FR-ACC-05, FR-DAT-08, FR-REV-04. Practice and the standard review must work without an account or network, because beginners should never be blocked from training by an online failure.

References

[1] Bertrand Meyer. *Handbook of Requirements and Business Analysis*. Springer, 2022.

[2] Ian Sommerville and Peter Sawyer. *Requirements Engineering: A Good Practice Guide*. Wiley, 1997.

[3] LuGus Studios. *Liftoff: FPV Drone Racing*. Official game features. Available: https://www.liftoff-game.com/liftoff-fpv-drone-racing. Accessed: October 18, 2026.

[4] VelociDrone. *VelociDrone FPV Simulator: Features*. Available: https://www.velocidrone.com/features. Accessed: October 18, 2026.

[5] LuGus Studios. *Liftoff XR Drone Released! (FREE)*. July 10, 2024. Available: https://www.liftoff-game.com/news/liftoffr-xr-drone-released-free. Accessed: October 18, 2026.

[6] Meta Platforms, Inc. *DVR Simulator*. Meta Quest Store. Available: https://www.meta.com/experiences/dvr-simulator/3744737298949811/. Accessed: October 18, 2026.

[7] Meta Platforms, Inc. *Drone Blitz*. Meta Quest Store. Available: https://www.meta.com/experiences/drone-blitz/24984912851173154/. Accessed: October 18, 2026.

[8] Unity Technologies. *Rigidbody Physics*. Unity 6.0 User Manual. Available: https://docs.unity3d.com/6000.0/Documentation/Manual/rigidbody-physics-section.html. Accessed: October 18, 2026.

[9] Meta Platforms, Inc. *Controller Input and Tracking Overview*. Meta Horizon OS Developer Documentation. Updated: February 13, 2026. Available: https://developers.meta.com/vr/documentation/unity/unity-ovrinput/. Accessed: October 18, 2026.

[10] Meta Platforms, Inc. *Age Group Self-Certification and Youth Requirements*. Meta Horizon OS Developer Documentation. Updated: April 2, 2026. Available: https://developers.meta.com/vr/resources/age-groups/. Accessed: October 18, 2026.

Appendix — Generative AI Use Disclosure

AI Tools Used

Claude Opus 5.5 and GPT Astra 6.1.

How and Where Used

1. **How were the AI tools used?**

   The AI tools were used as drafting, reviewing, and consistency-checking aids. They helped turn existing team decisions and source documents into draft text, compared sections of the SRS against each other and against the course rubric, and pointed out places where sections contradicted each other or where a requirement had no matching verification. Claude was also used to resolve merge conflicts between team members' branches and to make repetitive edits across the document, such as adding link targets to every requirement. The tools did not make project decisions: the headset, the scope of accounts and online results, which features are stretch goals, and the schedule were all decided by the team, and the AI-assisted text was written to match those decisions.

2. **Which parts of the document were affected by AI assistance?**

   - **Project book (P.1–P.7):** Drafting the roles, imposed technical choices, phase-in plan, tasks, technology elements, risks, and requirements process from the Development Plan, the capstone calendar, and the requirements in S.2. AI review also found dependency problems in the phase-in plan, such as requirements scheduled before the features they depend on.
   - **Likely and unlikely changes:** Drafting the description of how each likely change could happen and why each unlikely change is not expected, and linking each one to the affected requirements.
   - **Prioritization (S.5):** Checking that the phase-in plan used the priorities assigned in S.5.

- **Verification and acceptance criteria (S.6):** Drafting verification and validation entries for requirements that were added to S.2 after S.6 was first written, updating entries that no longer matched S.2, removing a duplicated copy of the section, and fixing broken links.
- **Functionality (S.2):** Rewording the hardware requirements to refer to the target headset, reference PCVR computer, and standard gamepad, with the specific devices recorded as constraints in P.2, and adding link targets to every requirement.
- **Limitations and Exclusions (G.6):** Rewording the multiplayer, historical records, and transmitter rows to match the team's stretch-goal decisions.
- **Other sections:** Team members used the tools for brainstorming and improving the clarity of wording in their own sections, in the same way as described in the Hazard Analysis.
- **Reflection:** Drafting parts of individual reflections from the author's description of their work, which the author then revised.

AI-generated suggestions were used as drafting and review aids rather than accepted automatically. The team remained responsible for reviewing the content and ensuring that the final document accurately reflected the FPVR project's scope, requirements, and design decisions.

Verification of AI-Generated Content

1. **How did the team check AI-generated or AI-assisted material before including it in the document?**

   Every AI-assisted change was reviewed by the team member responsible for that section before it was committed, and again through pull-request review by another team member before it was merged. Draft text was checked against its source: roles and tools against the Development Plan, milestone dates against the capstone calendar, and requirement references against S.2. When the AI suggested a scope or hardware change, such as whether user accounts were a stretch goal or which headset was the target, the decision was confirmed by the team rather

than taken from the suggestion. Several AI suggestions were rejected or reversed after review, for example an input-response target that was not backed by a requirement at the time it was proposed.

2. **How did the team verify factual claims, technical assertions, requirements, calculations, and references?**

   - **Requirement references:** Every requirement identifier mentioned in the Project book, S.5, and S.6 was checked against the identifiers defined in S.2, and the phase-in plan was checked to contain each requirement exactly once.
   - **Links:** The document was compiled with `pdflatex` after each change, and the build log was checked for errors, undefined references, and links to missing targets.
   - **Calculations:** Worked examples in S.6, such as the score for a given time and crash count under FR-PER-08, were checked by hand.
   - **Numerical targets:** Thresholds such as 72 FPS and 100 ms are treated as proposed targets to be confirmed on the target hardware (P.7), not as verified results.
   - **References:** AI tools were not used to produce citations. External references were found and checked by team members.

3. **Who is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document?**

   The team is ultimately responsible for the accuracy, completeness, consistency, and appropriateness of the submitted SRS, regardless of whether material was written by team members or generated with AI assistance. AI tools can help with drafting, reviewing, and finding inconsistencies, but their suggestions may be inaccurate, incomplete, or unsuitable for the project's scope. The team therefore critically evaluates AI-assisted content and ensures that the final document accurately represents the FPVR system and its requirements.

Appendix — Reflection

Arjun - Reflection

What went well while writing this deliverable?

Writing the Project book went well because most of its content could be grounded in sources the team had already agreed on. The roles in P.1 came from the Development Plan, the milestone dates in P.3 came from the capstone calendar, and the phase-in plan was built around those milestones, so every phase has a real due date instead of a guess. I also only referred to requirement IDs that already existed in S.2. This meant the phase-in plan, tasks, risks, and likely changes all point back to specific requirements, which makes the Project book traceable to the System book rather than a separate plan that could drift away from it.

What pain points did you experience during this deliverable, and how did you resolve them?

The main pain point was keeping the Project book consistent with sections that were being written at the same time. I first wrote the Project book when the SRS excluded networking and stored history, so P.2 said no network or database was needed, and the Development Plan and Hazard Analysis disagreed with that. While I was working, the System book was updated to add accounts, cloud results, leaderboards, and history, and to name the Quest 3S as the headset. When I merged those changes, my phase-in plan, technology list, and likely changes no longer matched the requirements. I resolved this by treating S.2 and S.5 as the source of truth: I added a phase for accounts and online results, used the S.5 priorities in the phase-in plan, listed the online service in P.2 and P.5, and kept only multiplayer and FPV transmitters as stretch goals. The lesson for me is that the Project book should be written after the requirements settle, or updated every time they change.

How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

The Project book does not define new requirements, but the plans in it were shaped by conversations. The supervisor meetings are the reason the flight model is phased in incrementally and the reference drone is listed as an open decision, and team discussions decided which features are stretch goals. None of the requirements came from speaking with potential end users, because no user consultation has happened yet (G.7). This is a weakness, and the elicitation plan in P.7 is meant to fix it before the requirements are accepted.

Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project?

SFWRENG 3RA3 (Software Requirements and Security Considerations) has helped the most so far. It taught me to write requirements that are verifiable, to separate imposed constraints from requirements, and to think about which requirements are likely or unlikely to change. These ideas map directly onto P.2, P.7, and the likely and unlikely changes in this document. The security part of the course is also directly relevant now that player accounts and online results are in scope, since requirements such as NFR-SEC-01 and NFR-SEC-02 need the same care in how credentials and access are specified.

What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project?

- **Unity and C# development** – the whole team, since every component is built in Unity.
- **VR development on the Meta Quest 3S with OpenXR** – John Cho and Ibrahim Tariq, for headset and controller integration and frame-rate performance.
- **Drone flight dynamics** – Ibrahim Tariq, to build a flight model that feels like a real FPV drone.

- **Level and environment design** – Sankalpa Chhetri Dhakal, for the obstacle courses and environments.
- **Machine learning for flight feedback** – Sankalpa Chhetri Dhakal, to evaluate whether the optional AI feedback is feasible.
- **State machine design and testing** – Vikram Chandar, to keep checkpoints, crashes, and timers consistent.
- **Scoring design and flight data recording** – Arjun Sasidaran, to design a score and rating that are fair and that users can understand.

For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

- **Unity and C#** – (1) Unity Learn tutorials and the Unity manual; (2) building small prototypes, such as a single checkpoint or timer, before the real features.
- **VR on the Meta Quest 3S** – (1) Meta's developer documentation and sample projects; (2) testing early builds on the headset and profiling them.
- **Drone flight dynamics** – (1) reading about quadcopter dynamics and the reference drone's specifications; (2) flying existing simulators such as Liftoff and comparing their behaviour.
- **Level design** – (1) studying the courses in existing FPV simulators; (2) playtesting courses with teammates and adjusting difficulty.
- **Machine learning** – (1) online courses and model documentation for pre-trained models such as Qwen2-VL; (2) testing a model on small samples of recorded flight data.
- **State machine design and testing** – (1) writing the state transitions down formally in the design document; (2) writing unit tests for each transition with Unity's test framework.

- **Scoring design and flight data recording** – (1) studying how existing simulators such as VelociDrone and Drone Blitz score and rank runs; (2) building a first version of the score, then calibrating it by having teammates fly the courses and checking whether the results match how well they actually flew.

For my own area, I will mainly pursue the second approach for scoring: building a first version and calibrating it with real runs. A scoring formula is hard to judge on paper, and seeing whether a clean run actually scores higher than a messy one is the quickest way to find out whether it is fair. I will still look at existing simulators first so that the first version starts from ideas that are known to work. For Unity and C#, I will build small prototypes, since I learn faster by writing code than by reading tutorials. The other team members describe their own choices in their reflections.

Sankalpa Chhetri Dhakal - Reflection

What went well while writing this deliverable?

The revision history records my adaptation of the SRS to Meyer's template and completion of Goals and S.2. This linked the project objectives to identifiable requirements and provided a common structure for the other books.

What pain points arose, and how can they be addressed?

Changes in S.2 can affect scope, environment, priorities and verification. Checking affected requirement IDs across these sections is necessary to keep the document consistent as the scope changes.

How did client or proxy discussions inspire the requirements?

The revision history does not establish how many requirements came from client or proxy discussions. G.7 records the available sources and consultation plans; a numerical attribution requires meeting records rather than an estimate.

Which courses or course topics support this work?

Requirements engineering and software design topics support testable requirements, clear boundaries and separating constraints from behaviour. Individual course enrolment is not established by the revision history.

What knowledge and skills does the team need?

The team needs Unity development, VR integration, flight dynamics, course design, state-based testing and scoring skills. My proposed focus is course design and evaluating optional intelligent feedback. The shared learning plan in Arjun's reflection identifies an area for every team member.

How will these skills be acquired, and which approach is proposed?

Two approaches for course design are studying existing FPV courses and running small playtests. I propose starting from examples and then prioritising playtests to expose unclear checkpoints and unsuitable difficulty. For optional feedback, reviewing model documentation and building a small feasibility prototype provide complementary approaches.

John Cho - Reflection

What went well while writing this deliverable?

The revision history records my initial content for S.1, S.3 and S.4. Describing components, interfaces and use cases together connected system responsibilities to observable interactions.

What pain points arose, and how can they be addressed?

Interfaces and use cases can drift when controller support or online services change. Comparing each exchange and use-case step with S.2 helps expose mismatches without prescribing internal implementation.

How did client or proxy discussions inspire the requirements?

The revision history does not establish how many requirements came from client or proxy discussions. G.7 records the available sources and consultation plans; a numerical attribution requires meeting records rather than an estimate.

Which courses or course topics support this work?

Software architecture, human-computer interaction and requirements engineering topics support component boundaries, usable VR interactions and

clear use cases. Individual course enrolment is not established by the revision history.

What knowledge and skills does the team need?

The team needs Unity development, VR integration, flight dynamics, course design, state-based testing and scoring skills. My proposed focus is Quest 3S integration, controller mappings and runtime behaviour. The shared learning plan in Arjun's reflection identifies an area for every team member.

How will these skills be acquired, and which approach is proposed?

Two approaches are reviewing runtime documentation and sample projects, and testing a minimal application on the headset. I propose beginning with a sample and then prioritising device tests, because tracking, input and connection problems need observation on the target hardware.

Vikram Chandar - Reflection

What went well while writing this deliverable?

The revision history records my completion of S.5 and S.6. Connecting priorities with verification and acceptance criteria made the implementation order and required evidence easier to review.

What pain points arose, and how can they be addressed?

A requirement may sound clear while lacking an observable pass condition. Checking each requirement against its verification method helps reveal missing thresholds, ambiguous outcomes and dependencies.

How did client or proxy discussions inspire the requirements?

The revision history does not establish how many requirements came from client or proxy discussions. G.7 records the available sources and consultation plans; a numerical attribution requires meeting records rather than an estimate.

Which courses or course topics support this work?

Software testing, requirements engineering and discrete mathematics topics support measurable acceptance criteria, traceability and state-transition checks. Individual course enrolment is not established by the revision history.

What knowledge and skills does the team need?

The team needs Unity development, VR integration, flight dynamics, course design, state-based testing and scoring skills. My proposed focus is state-machine testing for checkpoints, crashes and timers. The shared learning plan in Arjun's reflection identifies an area for every team member.

How will these skills be acquired, and which approach is proposed?

Two approaches are writing transition tables and implementing automated transition tests. I propose defining the expected transitions first, then prioritising tests for repeated crashes, skipped checkpoints and timer resets, since these cases can expose inconsistent state changes.

Ibrahim Tariq - Reflection

What went well while writing this deliverable?

The revision history records my completion of the Environment book. Separating components, constraints, assumptions, effects and invariants clarified the system boundary and the conditions needed for training.

What pain points arose, and how can they be addressed?

Accounts, cloud results and leaderboards required changes to the external-services table, context diagram and data rules. Keeping flow identifiers aligned and comparing Environment statements with S.2 helps prevent outdated exclusions.

How did client or proxy discussions inspire the requirements?

The revision history does not establish how many requirements came from client or proxy discussions. G.7 records the available sources and consultation plans; a numerical attribution requires meeting records rather than an estimate.

Which courses or course topics support this work?

Requirements engineering, mechanics and software testing topics support distinguishing assumptions from requirements, describing flight behaviour and planning repeatable performance checks. Individual course enrolment is not established by the revision history.

What knowledge and skills does the team need?

The team needs Unity development, VR integration, flight dynamics, course design, state-based testing and scoring skills. My proposed focus is simulated flight dynamics and performance on Quest 3S. The shared learning plan in Arjun's reflection identifies an area for every team member.

How will these skills be acquired, and which approach is proposed?

Two approaches are studying quadcopter dynamics and building small flight experiments. I propose using a simple model as a starting point, then prioritising repeatable flight tests and headset profiling to compare expected motion and frame delivery with observations.